

LONDON SOUTH BANK UNIVERISTY

DOCTORAL THESIS

Resilient Federated Learning to Manage Heterogeneity in Edge-Cloud Continuum

Author:

Satwat BASHIR

Supervisor:

Prof. Tasos DAGIUKLAS

*A thesis submitted in fulfillment of the requirements
for the degree of Doctor of Philosophy
in the*

SITHub Research Group

School of Computer Science and Digital Technologies

July 2026

LONDON SOUTH BANK UNIVERISTY

Abstract

School of Computer Science and Digital Technologies

Doctor of Philosophy

**Resilient Federated Learning to Manage Heterogeneity in Edge-Cloud
Continuum**

by Satwat BASHIR

The Thesis Abstract is written here (and usually kept to just this page). The page is kept centered vertically so can expand into the blank space above the title too.

Acknowledgements

The acknowledgments and the people to thank go here, don't forget to include your project advisor...

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Edge Intelligence and the Edge-Cloud Continuum	1
1.2 Federated Learning	2
1.2.1 From Centralised to Federated Training	3
1.2.2 The Federated Learning Objective	3
1.2.3 Federated Averaging	4
1.2.4 Three Properties of the Federated Learning Setting	5
1.3 Hierarchical Federated Learning in the Edge-Cloud Continuum	5
1.3.1 Hierarchical Objective and Two-Stage Aggregation	6
1.3.2 Multi-Timescale Operation	6
1.3.3 What HFL Handles, and What It Leaves Open	7
1.4 Research Challenges and Objectives	7
1.5 Contributions and Thesis Organisation	8
1.6 List of Publications	9
2 Background and Related Work	11
2.1 Heterogeneity in Hierarchical Federated Learning	11
2.1.1 Statistical Heterogeneity and Non-IID Data	11
2.1.2 Double Non-IID Structure in the Hierarchical Setting	12
2.1.3 System Heterogeneity and Partial Participation	12
2.1.4 The Per-Edge Personalisation Objective	12
2.2 Aggregation under Data Heterogeneity	13
2.2.1 Optimisation-Based Methods	13
2.2.2 Personalised Federated Learning	13
2.2.3 Clustered and Similarity-Based Aggregation	13
2.2.4 Aggregation in Hierarchical Federated Learning	14
2.3 Participant Selection for Efficiency	14
2.4 Orchestration and Resilience in the Edge-Cloud Continuum	14
2.5 Open Challenges	15
2.6 Summary	15
3 Fedge: A Bayesian Non-Parametric Hierarchical Federated Learning Framework	17
3.1 System Model and the Three-Way Trade-off	17
3.1.1 Three-Tier Hierarchical Setting	17
3.1.2 Two-Level Non-IID Heterogeneity Model	18
3.1.3 Partial Participation Model	18
3.1.4 The Three-Way Trade-off between Personalisation, Convergence, and Robustness	18

3.1.5	Output Specification	20
3.2	The Fedge Framework	20
3.2.1	The Variable Cloud-Tier Model Family	20
3.2.2	Dirichlet Process Prior over the Family	21
3.2.3	Soft Edge-to-Model Membership	21
3.2.4	Framework Properties	22
3.3	The Fedge Algorithm	23
3.3.1	Two-Level Heterogeneity Signals	23
3.3.2	Split-Merge Moves	23
3.3.3	Aggregation with Soft Membership	24
3.3.4	The Sensitivity Parameter	24
3.3.5	Algorithm Pseudocode	25
3.3.6	Computational and Communication Cost	25
3.4	Convergence Analysis	27
3.4.1	Assumptions	28
3.4.2	Stabilisation Window	29
3.4.3	Main Convergence Theorem	29
3.4.4	Proof Sketch	30
3.4.5	Implications for the Three-Way Trade-off	31
4	4	33
5	Orchestration for Resilient HFL	35
5.1	Problem Formulation	36
5.1.1	System Model	36
5.1.2	Training Objective and Aggregator Interface	37
5.1.3	Failure Model and Resilience Criterion	37
5.1.4	Orchestration as a Control Problem	38
5.2	Rule-based Fedge	39
5.2.1	Design Principle	40
5.2.2	Architecture	40
5.2.3	Per-Round Control Loop	42
5.3	Agentic Fedge	42
5.3.1	Design Principle	42
5.3.2	Architecture	43
5.3.3	Per-Round Control Loop	46
5.4	Experimental Evaluation	46
5.4.1	Experimental Setup	46
5.4.2	Within-Partition Recovery	49
5.4.3	Cross-Partition Failover	51
5.4.4	Tier-3 Memory Ablation	53
5.5	Summary	56
6	Conclusion	57
6.1	Limitations and Future Work	57
6.1.1	Chapter 5: Agentic Orchestration	57
A	Full Proof of the Fedge Convergence Theorem	59
A.1	One-Step Descent Inequality	59
A.2	Bound on Local-Update Drift	60
A.3	Variance of the Membership-Weighted Aggregate	60

A.4 Telescoping over Post-Warm-Up Rounds	61
A.5 Notes on the Constants	62

List of Figures

1.1	The edge-cloud continuum. Compute and storage capacity increase from the device tier to the cloud tier, while proximity to data sources and round-trip latency move in the opposite direction. Each tier offers a different trade-off between locality and capability, and federated learning systems are typically deployed across all four.	2
1.2	Centralised ML versus federated learning. (a) In the centralised paradigm, raw data is transmitted from clients to a central server, where it is accumulated and used for training. (b) In federated learning, raw data is retained at the client; clients perform local training and transmit only model updates to the server, which aggregates the updates into a new global model and redistributes it for the next round.	3
1.3	Three-tier hierarchical federated learning. Clients are partitioned across edge servers; each edge aggregates updates from its own client subset every τ_1 local steps. The cloud aggregates over the edge servers every τ_2 edge rounds, with $\tau_2 \gg \tau_1$	6
3.1	The three-way trade-off in hierarchical federated learning	20
3.2	The Fedge architecture	21
5.1	Rule-based Fedge architecture	41
5.2	Agentic Fedge architecture	44
5.3	Scenario A: accuracy gap from Clean under failure	51
5.4	Tier-1 fallback rate across scenarios	53
5.5	Tier-3 memory ablation	55

List of Tables

3.1	Per-round cost comparison for the Fedge algorithm against four hierarchical federated learning baselines. Notation as in Section 3.3.6. The relative total-bits multiplier is the ratio of total transmitted bits relative to HierFAVG under the same number of cloud rounds T . The MTGC upload and download figures are worst-case; Fang <i>et al.</i> [34] note that control variates can be estimated from model differences with reduced transmission cost at the price of persistent per-client and per-edge storage. The ESPerHFL row reports edge–cloud cost; client–edge cost is higher due to per-client personalisation parameters.	28
5.1	Scenario A operational metrics at medium failure intensity (MTBF = 570, $\kappa_e = 20$). Mean $\pm$ standard deviation over independent runs. Bold marks the best value per column among the failover methods.	49
5.2	Scenario A operational metrics at high failure intensity (MTBF = 250, $\kappa_e = 25$). Mean $\pm$ standard deviation over independent runs.	50
5.3	Scenario B operational metrics (cluster cascade, $\kappa_f^{\text{edge}} = 8$, $T = 200$ rounds). Mean $\pm$ standard deviation over independent runs. Bold marks the best value per column among the capacity-respecting methods.	52
5.4	Scenario B agentic-specific diagnostics (mean $\pm$ standard deviation over independent runs).	52
5.5	Tier-3 memory ablation (Scenario A, high intensity, $T = 200$ rounds; ten memory-on runs, three memory-off runs). Mean $\pm$ standard deviation per arm; the rightmost column reports the per-metric difference $\Delta = (\text{off} - \text{on})$. Bold rows mark the metrics on which the two arms separate beyond noise.	54

List of Abbreviations

LAH List Abbreviations **Here**
WSF What (it) Stands For

Physical Constants

Speed of Light $c_0 = 2.997\,924\,58 \times 10^8 \text{ m s}^{-1}$ (exact)

List of Symbols

a	distance	m
P	power	W (J s ⁻¹)
ω	angular frequency	rad

For/Dedicated to/To my...

Chapter 1

Introduction

1.1 Edge Intelligence and the Edge-Cloud Continuum

The data on which modern machine learning (ML) systems depend is increasingly generated outside the data centres in which those systems have traditionally been trained and operated. Smartphones, vehicles, industrial sensors, wearable health monitors, and the wider population of devices that constitute the Internet of Things (IoT) now produce a substantial and growing share of the data that ML algorithms consume, and this share is expected to continue rising as connected devices proliferate [1–3]. Under the conventional cloud-centric pipeline, raw data is transported from its source to a centralised facility, processed there, and the results are returned to the device. This arrangement remains effective for tasks that tolerate latency and operate on aggregated or non-sensitive data, but it is no longer the only viable architecture, and in a growing number of settings it is not the preferred one [4, 5].

Four pressures account for the shift of computation and learning toward the edge of the network. These pressures are well established in the edge computing literature [1–3] and motivate the architectural setting in which the contributions of this thesis are situated.

The first pressure is latency. Applications such as autonomous driving, augmented reality, and industrial control require response times measured in milliseconds, which cannot be met by a round-trip to a distant cloud facility [2, 6]. The end-to-end latency of a wide-area request includes propagation delay, queuing at intermediate routers, and processing at the cloud, and is dominated by the propagation component when the cloud is hundreds of kilometres from the data source. Placing computation closer to the source removes the wide-area network from the critical path and bounds the round-trip delay by the local access network alone [2, 4].

The second pressure is bandwidth. Sensor streams, video feeds, and high-frequency telemetry generate volumes of data that are inefficient to transport in raw form when only a small fraction of the derived information is needed downstream. When raw data is processed near its source, redundant content can be filtered, summarised, or aggregated before transmission, and the load on the backhaul link is reduced accordingly [1, 3, 7]. This argument has been made repeatedly in the context of the Internet of Things, where the cumulative output of a large population of devices can exceed the capacity of the network that would otherwise carry it [4].

The third pressure is privacy. Data-protection regulations, in particular the General Data Protection Regulation in the European Union [8], restrict the movement of personal data and impose obligations on controllers that are difficult to satisfy when raw data is centralised across jurisdictions. Beyond legal constraints, users and institutions are increasingly reluctant to surrender raw data to remote operators, particularly when the data concerns health, location, or behaviour [9]. Keeping

FIGURE 1.1: The edge-cloud continuum. Compute and storage capacity increase from the device tier to the cloud tier, while proximity to data sources and round-trip latency move in the opposite direction. Each tier offers a different trade-off between locality and capability, and federated learning systems are typically deployed across all four.

data close to its source reduces the attack surface over which sensitive content is exposed and limits the number of parties that must be trusted.

The fourth pressure is resilience. Concentrating computation in a small number of central facilities creates correlated failure modes, in which the loss of a single facility can disrupt services across a wide area. A distributed substrate, by contrast, can continue to serve workloads in the affected region from neighbouring nodes, and intermittent connectivity between tiers can be tolerated rather than treated as a fatal condition [2, 4]. Edge nodes are not individually more reliable than cloud facilities. The benefit instead is that the failure of any single node has a smaller blast radius when the workload is distributed across many nodes that share no common dependency.

These four pressures have given rise to a layered computing substrate that spans the path from end devices to centralised cloud resources. This substrate is referred to in the literature as the edge-cloud continuum: a tiered infrastructure comprising end-user devices at one extreme, edge servers at the network boundary, regional aggregation or fog nodes at intermediate locations, and cloud data centres at the other extreme [4, 5, 7]. As one moves from the device tier toward the cloud, available compute and storage capacity increase, while proximity to data sources decreases and round-trip latency grows [3]. The continuum is therefore not a uniform pool of resources but a tiered one, in which each tier presents a different trade-off between locality and computational capacity. Figure 1.1 illustrates the tiered structure of the continuum and the characteristic properties of each tier.

Edge intelligence has been discussed under a variety of adjacent labels in the literature, including mobile edge computing, fog computing, and AI-native networks in the wireless community [3], and the on-device deployment of compact models is the subject of a parallel body of work on TinyML and embedded inference. This thesis is concerned with learning algorithms and orchestration mechanisms operating within the edge-cloud continuum, and does not contribute at the wireless layer or at the level of on-device model compression. The continuum is taken as the deployment substrate; the question that follows is how machine learning models can be trained on data that is distributed across this substrate and cannot, by virtue of the four pressures above, be collected centrally. This question is the subject of the next section.

1.2 Federated Learning

The four pressures established in Section 1 produce a concrete operational constraint for ML: the data on which a model must be trained is held by clients distributed across the edge-cloud continuum and cannot, for reasons of latency, bandwidth, privacy, or regulation, be transferred to a central server for processing. Conventional ML pipelines do not accommodate this constraint. They assume that training data is co-located with the training process and that a single optimisation procedure has unrestricted access to the entire dataset. Federated learning (FL) is the ML paradigm

FIGURE 1.2: Centralised ML versus federated learning. (a) In the centralised paradigm, raw data is transmitted from clients to a central server, where it is accumulated and used for training. (b) In federated learning, raw data is retained at the client; clients perform local training and transmit only model updates to the server, which aggregates the updates into a new global model and redistributes it for the next round.

developed to address this constraint by reversing the data flow: instead of bringing the data to the model, the model is brought to the data [9, 10].

1.2.1 From Centralised to Federated Training

Under the centralised ML paradigm illustrated in Figure 1.2(a), training proceeds in two stages. First, raw data from a population of sources is transmitted to a central server, where it is accumulated into a single dataset. Second, the server runs a training procedure, typically stochastic gradient descent (SGD) or one of its variants, over the accumulated dataset to obtain a trained model. The server retains the data, the training process, and the resulting model. Clients act as data sources and, optionally, as recipients of the trained model at inference time. This arrangement is efficient when the data can be moved freely and the cost of moving it is acceptable, but it is unable to satisfy the four pressures described in Section 1.1.

Under the FL paradigm illustrated in Figure 1.2(b), the data is never transmitted. Each client retains its local dataset and performs local training on its own data. Clients transmit only the resulting model updates to a central server, which aggregates the updates from the participating clients to produce a new global model. The new global model is then redistributed to the clients for the next round of local training. This process is repeated for a sequence of communication rounds until the global model converges. Raw data remains on the device throughout; only model parameters cross the network [9, 10].

1.2.2 The Federated Learning Objective

FL was introduced by McMahan et al. [10] in the context of on-device language modelling for mobile keyboards, and the formal objective they introduced is the basis for most subsequent work in the area. Consider a population of K clients, where each client $k \in \{1, \dots, K\}$ holds a local dataset $\mathcal{D}_k$ drawn from a client-specific distribution. Let $F_k(w)$ denote the local empirical risk of model parameters $w \in \mathbb{R}^d$ on client k , defined as

$$F_k(w) = \frac{1}{|\mathcal{D}_k|} \sum_{\xi \in \mathcal{D}_k} \ell(w; \xi), \quad (1.1)$$

where $\ell(w; \xi)$ is the per-sample loss of model w on training sample ξ . The federated learning objective is to find parameters w^* that minimise the weighted average of local risks across all clients:

$$\min_{w \in \mathbb{R}^d} F(w) = \sum_{k=1}^K \frac{|\mathcal{D}_k|}{|\mathcal{D}|} F_k(w), \quad (1.2)$$

Algorithm 1 Federated Averaging (FedAvg) [10]

Require: Number of rounds T ; local epochs E ; local mini-batch size B ; local learning rate η ; client fraction $C \in (0, 1]$

```

1: Server initialises global model  $w^{(0)}$ 
2: for each communication round  $t = 0, 1, \dots, T - 1$  do
3:   Server selects a subset  $S_t$  of  $\max(C \cdot K, 1)$  clients
4:   Server broadcasts  $w^{(t)}$  to each client  $k \in S_t$ 
5:   for each client  $k \in S_t$  in parallel do
6:      $w_k^{(t+1)} \leftarrow \text{LOCALSGD}(k, w^{(t)}, E, B, \eta)$ 
7:     Client  $k$  transmits  $w_k^{(t+1)}$  to the server
8:   end for
9:   Server aggregates:  $w^{(t+1)} \leftarrow \sum_{k \in S_t} \frac{|\mathcal{D}_k|}{\sum_{j \in S_t} |\mathcal{D}_j|} w_k^{(t+1)}$ 
10: end for
11: return  $w^{(T)}$ 

```

```

12: function LOCALSGD( $k, w, E, B, \eta$ )
13:   for each local epoch  $e = 1, \dots, E$  do
14:     for each mini-batch  $b$  of size  $B$  from  $\mathcal{D}_k$  do
15:        $w \leftarrow w - \eta \nabla \ell(w; b)$ 
16:     end for
17:   end for
18:   return  $w$ 
19: end function

```

where $|\mathcal{D}| = \sum_{k=1}^K |\mathcal{D}_k|$ is the total dataset size across all clients. The weighting by $|\mathcal{D}_k|/|\mathcal{D}|$ ensures that the federated objective reduces to standard empirical risk minimisation when data is identically distributed across clients and all data is available centrally [9].

1.2.3 Federated Averaging

The canonical algorithm for solving the FL objective in Equation (1.2) is Federated Averaging (FedAvg), introduced in the same paper as the objective itself [10]. FedAvg proceeds in synchronous communication rounds. In each round t , the server selects a subset of clients to participate, broadcasts the current global model $w^{(t)}$ to the selected clients, and each selected client performs E epochs of local SGD on its own data. The clients then return their updated parameters to the server, which computes the next global model $w^{(t+1)}$ as the data-size-weighted average of the returned parameters. The procedure is summarised in Algorithm 1.

The key design choice in FedAvg is the use of multiple local SGD steps per communication round, controlled by the local epoch count E . Setting $E = 1$ with full-batch local gradients recovers parallel SGD with synchronous gradient aggregation, in which case the algorithm communicates after every gradient computation. Setting $E > 1$ allows clients to perform substantial local work between communication rounds, which is what makes the per-round communication cost tractable in settings where the wide-area uplink is the dominant bottleneck [9, 10]. The trade-off introduced by larger E is statistical: when client data is non-identically distributed, more local steps cause client models to drift toward their individual local minima rather than the global optimum, an effect known as client drift.

1.2.4 Three Properties of the Federated Learning Setting

The FL setting differs from centralised ML and from classical distributed training in three respects that together define the technical difficulty of the area [9–11].

Distributed parallel computation under unreliable, non-uniform clients. FL clients communicate over high-latency, low-bandwidth wide-area links rather than the high-bandwidth interconnects available in a data centre. Participation is intermittent: clients drop, reconnect, and vary in availability over time. Client compute and bandwidth capabilities also differ by orders of magnitude across the population. This setting is qualitatively different from data-centre distributed training, where the cluster is homogeneous, reliable, and centrally administered, and is one of the principal sources of systems difficulty in deploying FL at scale.

Strict data locality. Raw client data does not leave the client. Only model parameters or gradients are transmitted to the server. This property is the foundation of the privacy argument for FL and is the reason the paradigm responds directly to the third pressure in Section 1.1. Strict data locality does not, on its own, provide formal privacy guarantees: transmitted updates can in principle leak information about the underlying data through gradient-leakage attacks [12], and a body of work on privacy-preserving aggregation has developed in response. This thesis does not contribute at the level of privacy-preserving aggregation, and treats data locality as the operational property that distinguishes FL from centralised training.

Heterogeneous, non-IID client data. Each client’s local dataset is drawn from a client-specific distribution rather than from a shared global distribution. Aggregated across the population, the data is non-independent and non-identically distributed (non-IID). Client distributions can also drift over time as user behaviour, sensor placement, or environmental conditions change. This is unlike data-centre training, where data is typically shuffled and partitioned IID across workers. Non-IID data is the principal source of statistical difficulty in FL and produces measurable degradation in convergence rate and final model accuracy under FedAvg [9, 11]. Together with the resource heterogeneity introduced by non-uniform client capabilities, it is the heterogeneity dimension that the contributions of this thesis address.

A further question remains: the formulation above uses only two of the four tiers introduced in Section 1.1, leaving the intermediate tiers idle and the wide-area uplink in the critical path of every round.

1.3 Hierarchical Federated Learning in the Edge-Cloud Continuum

The two-tier topology assumed by Equation (1.2) and Algorithm 1 is at odds with the four-tier substrate established in Section 1.1. Every client update has to traverse the wide-area uplink to reach the cloud, which reintroduces the latency and bandwidth costs that the continuum was supposed to absorb. The cloud server becomes the single point of coordination for the entire population, which is inconsistent with the resilience argument made earlier. Further, the rate at which coordination can react to system events is bounded by the round-trip to the cloud, even though most events of interest, such as a client joining, leaving, or failing, occur much closer to the edge.

FIGURE 1.3: Three-tier hierarchical federated learning. Clients are partitioned across edge servers; each edge aggregates updates from its own client subset every τ_1 local steps. The cloud aggregates over the edge servers every τ_2 edge rounds, with $\tau_2 \gg \tau_1$.

Hierarchical federated learning (HFL) closes this gap by placing intermediate aggregators between the clients and the cloud. The canonical form, introduced by Liu et al. [13] and now standard for FL on the edge-cloud continuum [14, 15], is the three-tier client–edge–cloud architecture shown in Figure 1.3. Each edge server serves a disjoint subset of clients and performs partial aggregation over the updates it receives; the cloud performs a second stage of aggregation over the edge-aggregated models. The same three logical roles can be mapped onto deeper physical infrastructures, such as base stations, on-premises servers, and regional fog nodes, all of which serve the edge role in the algorithmic sense.

1.3.1 Hierarchical Objective and Two-Stage Aggregation

Let $\mathcal{E} = \{1, \dots, M\}$ denote the set of edge servers and $\mathcal{K}_e$ the disjoint client subset served by edge e , so that $\bigcup_{e \in \mathcal{E}} \mathcal{K}_e = K$ and $M \ll K$. The hierarchical objective is

$$\min_{w \in \mathbb{R}^d} F(w) = \sum_{e \in \mathcal{E}} \sum_{k \in \mathcal{K}_e} \frac{|\mathcal{D}_k|}{|\mathcal{D}|} F_k(w), \quad (1.3)$$

which reduces to Equation (1.2) when $M = 1$. The hierarchical formulation therefore extends flat FL rather than replacing it.

Aggregation is performed in two stages. Let $\mathcal{A}$ denote the aggregation operator, instantiated as data-size-weighted averaging in the canonical case. At the edge tier,

$$w_e = \mathcal{A}(\{w_k\}_{k \in \mathcal{K}_e}), \quad (1.4)$$

and at the cloud tier,

$$w = \mathcal{A}(\{w_e\}_{e \in \mathcal{E}}). \quad (1.5)$$

This pair of operators, applied at the frequencies described next, gives the hierarchical FedAvg algorithm (HierFAVG) [13], which is the standard baseline for HFL.

1.3.2 Multi-Timescale Operation

The two aggregation stages run at different rates. Edge aggregation, Equation (1.4), is performed every τ_1 local SGD steps; cloud aggregation, Equation (1.5), is performed every τ_2 edge rounds, with $\tau_2 \gg \tau_1$. The inner loop runs at a high frequency over a short-haul link; the outer loop runs at a lower frequency over the wide-area backhaul. This separation is the source of the communication efficiency that distinguishes HFL from flat FL: aggregation traffic is absorbed at the edge tier wherever possible, and only periodic edge-aggregated updates cross the wide-area network [13, 15].

The trade-off introduced by τ_2 is statistical. Larger τ_2 reduces wide-area communication but allows more local drift between cloud aggregations, and the drift is amplified by the non-IID property identified in Section 1.2. The communication–convergence trade-off under non-IID data, and the staleness and association decisions that interact with it, are an active line of work in HFL [14, 15].

1.3.3 What HFL Handles, and What It Leaves Open

The mechanisms above explain why HFL is the standard realisation of FL on the edge-cloud continuum. Multi-timescale aggregation keeps most coordination traffic short-haul and removes the cloud from the per-round critical path, which improves both communication efficiency and resilience without weakening the data locality property that distinguishes FL from centralised training. The hierarchical structure additionally provides a place to absorb client heterogeneity, since clients can be assigned across edges in alignment with the similarity of their local data distributions [16], or with geographic proximity or device capability.

These properties are all properties of the algorithmic layer of HFL: how aggregation proceeds once the system is configured and operating normally. The control layer of HFL has received considerably less attention. In the canonical formulation [13], the edge partition $\{\mathcal{K}_e\}_{e \in \mathcal{E}}$ is fixed in advance, participant selection within each edge is uniform random, edges are assumed to remain available, and the cloud is assumed reachable and authoritative. These assumptions are convenient for algorithmic analysis but unrealistic for a deployment on the edge-cloud continuum, where edges fail, client populations shift, data distributions drift across partitions, and the wide-area link to the cloud is intermittent. The control layer of HFL, which determines how clients are assigned to edges, how participants are selected, and how the system reacts when components fail, is the open problem that the rest of this thesis addresses. The next section formulates this problem as three specific research challenges and the corresponding thesis objectives.

1.4 Research Challenges and Objectives

Although HFL extends FL to the edge-cloud continuum, three challenges remain unaddressed in the canonical HFL formulation. These challenges arise in deployment rather than in algorithmic analysis, and together account for the gap between HFL as described in the literature and HFL as it must operate in practice.

The first challenge concerns aggregation. Canonical HFL applies data-size-weighted averaging at both the client-to-edge and the edge-to-cloud tiers, and treats clients within an edge, and edges within the cloud, as statistically uniform. Under non-IID data this assumption is violated at both tiers, and uniform weighting amplifies the client drift identified in Section 1.2. Existing aggregation modifications such as proximal regularisation and variance reduction operate on flat FL and do not separate the two tiers of heterogeneity that arise in the hierarchical setting.

The second challenge concerns participant selection. Canonical HFL selects participating clients within each edge uniformly at random. Under non-IID data this is inefficient: uniform random selection ignores how much each client's update contributes to convergence, so communication and computation are spent on updates that advance the global model little. Most selection methods proposed in the FL literature operate on flat FL and do not exploit the hierarchical structure, in which a selection decision is taken at the edge and a complementary decision is taken at the cloud.

The third challenge concerns failure handling. Canonical HFL assumes that edges remain available throughout training, that the cloud is reachable, and that the partition is static. In a deployment these assumptions do not hold: edges fail, client populations shift, data distributions drift across partitions, and connectivity to the cloud is intermittent. HFL papers concentrate on the algorithmic behaviour and leave failure handling implicit, with no defined behaviour beyond restart.

The aim of this thesis is to develop new algorithms and orchestration mechanisms for HFL that address aggregation under non-IID data, participant selection for training efficiency under non-IID data, and failure handling during training on the edge-cloud continuum. The specific objectives are:

1. To develop a new aggregation algorithm for HFL that accounts for non-IID data at both the client-to-edge and edge-to-cloud tiers, and to evaluate its effect on convergence rate and final accuracy.
2. To develop a new participant-selection strategy that improves training efficiency under non-IID data by preferentially selecting informative participants rather than selecting uniformly at random, and that improves convergence over uniform random selection without assuming static participant assignments.
3. To develop a new orchestration mechanism that maintains training continuity under edge and cloud failures on the edge-cloud continuum, with explicit failure handling and auditable decisions.

The next section describes the contributions made against these objectives and the organisation of the remainder of the thesis.

1.5 Contributions and Thesis Organisation

To achieve the research objectives set out in Section 1.4, the remaining chapters of this thesis make the following original contributions.

Chapter 2 provides a detailed and formal description of HFL and the wider FL setting on which it is built, and surveys important developments in aggregation, participant selection, and orchestration that are built upon in the other chapters. It also overviews related work in edge computing and distributed training that is relevant to this thesis. Current challenges to the state of the art are also presented.

Chapter 3 develops Fedge, a novel framework for personalised HFL under non-IID data. The framework operates at the cloud tier through three complementary mechanisms: similarity-weighted personalised aggregation that produces a distinct model for each edge based on functional similarity between edge updates, an adaptive temperature controller that automatically balances knowledge sharing and specialisation as training progresses, and a global knowledge fusion component that prevents over-specialisation. This approach reduces client drift, accelerates convergence, improves final accuracy, and narrows the generalisation gap between the global model and client models compared with flat-FL and traditional HFL baselines.

Chapter 4 proposes Fed-BAC, a new participant-selection strategy for HFL that improves training efficiency under non-IID data by exploiting the hierarchical structure. A two-level bandit framework is introduced in which participant selection at the edge and the complementary selection decision at the cloud are learned jointly during training, rather than fixed in advance. Extensive experiments on three benchmark datasets show substantial improvements in distributed accuracy, faster convergence, lower cross-server variance, and reduced communication cost compared with canonical HFL and uniform-random-selection baselines, with the advantage widening as data heterogeneity becomes more severe.

Chapter 5 presents a novel orchestration approach for HFL on the edge-cloud continuum that maintains training continuity under failure. Two complementary

control-plane designs are developed: a deterministic rule-based design with a pre-computed plan, and an agentic design built around a large-language-model deliberative layer behind a deterministic safety floor that guarantees correctness. The orchestration handles edge-level failures and cluster-level failover within a single architectural template, preserves training progress without restart under repeated component failures, and provides a per-decision audit log of all coordination actions.

Chapter 6 concludes the thesis. After reviewing the technical contributions made in the previous chapters, current challenges to the state of the art and future directions within HFL on the edge-cloud continuum are discussed.

1.6 List of Publications

Chapter 2

Background and Related Work

Chapter 1 established the formal setting of this thesis: the federated learning objective of Equation (1.2), its hierarchical extension in Equation (1.3), the two-stage aggregation of Equations (1.4) and (1.5), and the three deployment challenges that the canonical formulation leaves open. The present chapter examines why those challenges are difficult and what has been proposed against them. The heterogeneity that underlies all three challenges is characterised first, in a form that the contribution chapters reuse without restating. The developments that each contribution builds upon are then surveyed in turn, in the order in which the contributions appear, and the chapter closes by stating the specific gaps that motivate Chapters 3–5. Related work in edge computing and distributed training is treated where it bears on these gaps rather than as a separate survey, consistent with the edge-cloud continuum setting fixed in Chapter 1.

2.1 Heterogeneity in Hierarchical Federated Learning

The difficulty of federated learning on the edge-cloud continuum is largely a consequence of heterogeneity. This section distinguishes its forms, formalises the structure that is specific to the hierarchical setting, and states the per-edge personalisation objective. The definitions introduced here are referred to, but not repeated, in the contribution chapters.

2.1.1 Statistical Heterogeneity and Non-IID Data

Let client k hold data drawn from a client-specific distribution $\mathcal{P}_k$ over the input-label space. In the federated setting these distributions are not identical across clients, so the aggregated population is non-independent and non-identically distributed (non-IID). The discrepancy between $\mathcal{P}_k$ and the population distribution arises from label skew, feature skew, and quantity skew, and from the joint forms in which these occur in practice [9]. The effect of non-IID data on Federated Averaging has been measured directly: accuracy on skewed partitions falls well below the identically distributed case, and the degradation grows with the degree of skew [17, 18]. The mechanism is the client drift identified in Section 1.2.3: when local distributions differ, the local objectives in Equation (1.1) have displaced minima, so local updates move toward client-specific optima rather than the federated optimum, and their average is biased relative to the objective of Equation (1.2). Statistical heterogeneity is the principal source of difficulty addressed by the first contribution of this thesis.

2.1.2 Double Non-IID Structure in the Hierarchical Setting

The hierarchical objective of Equation (1.3) introduces a structure that flat federated learning does not possess: heterogeneity occurs at two tiers rather than one. Within an edge, the client subset $\mathcal{K}_e$ is itself non-IID, so the edge-aggregated model w_e of Equation (1.4) is subject to client drift in the same way as a flat model. Between edges, the edge subsets are drawn from different regions of the population, so the edge models $\{w_e\}_{e \in \mathcal{E}}$ presented to the cloud aggregation of Equation (1.5) are themselves non-identically distributed. This two-tier, or double non-IID, structure means that a single cloud-aggregated model redistributed unchanged to every edge is a compromise that fits no edge well. Canonical hierarchical Federated Averaging applies data-size-weighted averaging at both tiers and treats clients within an edge, and edges within the cloud, as statistically uniform [13]; the assumption is violated at both tiers under non-IID data. The double non-IID structure is the object that the first contribution addresses directly.

2.1.3 System Heterogeneity and Partial Participation

In addition to statistical heterogeneity, clients differ in compute, memory, and network capability by orders of magnitude, and their availability varies over time [9, 11]. As a consequence, only a fraction of the clients associated with an edge participates in any given round; this participating fraction, denoted ρ , is typically small in deployment because clients are constrained by connectivity, battery, and contention. Partial participation has two effects that matter for the contributions. First, it makes the edge-level signal noisy, because the participating subset changes between rounds and is not representative of the edge distribution. Second, it makes computation and communication a scarce resource that is wasted when spent on participants whose updates contribute little to convergence. Robustness to this combination of statistical and system heterogeneity remains an open problem in the recent literature [19, 20]. Partial participation under non-IID data is the setting in which the second contribution operates.

2.1.4 The Per-Edge Personalisation Objective

The double non-IID structure motivates a departure from the single-global-model objective. Rather than seeking one model w that minimises Equation (1.3) for all edges, a distinct model $\tilde{w}_e$ is sought for each edge e , such that $\tilde{w}_e$ performs well on the distribution of $\mathcal{K}_e$ while still benefiting from knowledge held by edges with similar distributions. This per-edge personalisation objective trades two failure modes against each other. Averaging too strongly across dissimilar edges reproduces the single-global-model compromise and discards edge-specific structure; isolating each edge entirely forgoes the cross-edge knowledge transfer that accelerates convergence and improves accuracy when edges are related. The balance between these modes is not fixed during training: broad sharing is beneficial early, when edge models are poorly determined, and progressively less beneficial later, as edge-specific structure is acquired. This objective, and the trade-off it expresses, is the formal basis of the first contribution and is referred to from Chapter 3 without restatement.

2.2 Aggregation under Data Heterogeneity

The first body of related work concerns how model updates are combined under non-IID data. Methods are grouped into optimisation-based correction, personalisation, similarity-based aggregation, and aggregation specific to the hierarchical setting. The first contribution of this thesis is positioned against the last two.

2.2.1 Optimisation-Based Methods

One line of work modifies the local objective or the aggregation rule to reduce client drift while retaining a single global model. FedProx adds a proximal term that penalises divergence of the local model from the current global model [21]. SCAFFOLD maintains control variates that estimate and correct the difference between local and global update directions [22]. FedNova normalises updates to remove the objective inconsistency introduced by uneven local progress [23], FedDyn introduces a dynamic regulariser that aligns local and global stationary points [24], and adaptive server optimisation applies adaptive optimisers at the aggregation step [25]. These methods reduce drift but operate within a flat, single-tier topology and produce a single shared model; they do not separate, or exploit, the two tiers of heterogeneity identified in Section 2.1.2.

2.2.2 Personalised Federated Learning

A second line of work abandons the single-model assumption and produces client-specific models. pFedMe formulates personalisation through Moreau envelopes, decoupling the personalised model from the global model with a regularised local objective [26]. Per-FedAvg casts personalisation as meta-learning, training an initialisation that adapts quickly to each client [27]. Federated multi-task learning treats each client as a related task [28], partial model personalisation shares only a subset of parameters [29], and FedBN keeps normalisation statistics local to each client [30]. These methods personalise in a flat client-server topology and typically assume full participation; they do not address the edge tier introduced by the hierarchical objective, nor the unreliability of the edge signal under partial participation.

2.2.3 Clustered and Similarity-Based Aggregation

Closest to the first contribution is work that aggregates selectively according to similarity between participants. Clustered federated learning partitions clients into groups and trains one model per group, using parameter-space or gradient-space similarity to assign membership [31]. The iterative clustered framework alternates between cluster assignment and model estimation [32], and soft clustering relaxes the assignment so that a client may contribute to several clusters with graded weight [33]. Hierarchical clustering of local updates has also been used to separate incompatible clients before aggregation [16]. These methods establish that similarity-aware aggregation improves training under non-IID data, but they rely on a fixed or iteratively re-estimated number of groups and on similarity measured in parameter or gradient space, which reflects how models are configured rather than how they behave. The first contribution departs from this line by measuring similarity functionally, on model outputs, and by weighting contributions continuously rather than assigning discrete membership; the relationship is made precise in Chapter 3.

2.2.4 Aggregation in Hierarchical Federated Learning

Hierarchical Federated Averaging established the two-stage aggregation of Equations (1.4) and (1.5) as the standard baseline [13]. Subsequent work has refined the hierarchy along several axes: HiFlash adds adaptive staleness control and heterogeneity-aware client–edge association [14], multi-timescale gradient correction stabilises training across the two timescales under non-IID data [34], and architectural studies map the three tiers onto deployed infrastructure [15]. Personalisation has more recently been introduced into the hierarchy: personalised client–edge–cloud learning aligns edge models before redistribution [35], AdaptiveHierFL adjusts tiers and transfers knowledge across them [36], hierarchical personalisation has been studied specifically under non-IID edge data [37], and hierarchical aggregation has been tailored to heterogeneous IoT scenarios [38]. These methods either retain a single redistributed model or determine the degree of personalisation by a fixed or manually scheduled rule, and they assume reliable, full edge participation. The first contribution addresses precisely this combination, and is positioned against these methods in Chapter 3.

2.3 Participant Selection for Efficiency

The second body of related work concerns which participants contribute in a given round. Canonical hierarchical Federated Averaging selects participating clients within each edge uniformly at random [13]. Under the non-IID and partial-participation conditions of Section 2.1.3, uniform random selection is inefficient: it ignores how much a given participant’s update advances the federated objective, so scarce communication and computation are spent on updates that contribute little. Selection strategies that bias participation toward more informative clients have been shown to improve convergence per unit of communication, and the design space, its principles, and its open problems have been surveyed in detail [39, 40]. Representative methods select clients to maximise an efficiency criterion under resource constraints [41], and context-aware online selection has been studied for the hierarchical setting specifically [42]. Most of these methods, however, act at a single tier and either assume a static client-to-edge assignment or treat selection independently of the hierarchical structure, in which a selection decision is taken at the edge and a complementary decision is taken at the cloud. The second contribution addresses participant selection for training efficiency that exploits this two-level structure and adapts during training, and is positioned against these methods in Chapter 4.

2.4 Orchestration and Resilience in the Edge-Cloud Continuum

The third body of related work concerns how the system is coordinated and how it behaves when components fail. Surveys of the edge-cloud continuum establish that intermittent connectivity between tiers and the loss of individual nodes are expected operating conditions rather than exceptional ones [4, 5]. Within hierarchical federated learning, orchestration has been studied as a resource and communication problem: reactive orchestration under a communication-cost budget [43] and inference-load-aware orchestration [44] adjust the configuration of the hierarchy during operation, and an architectural blueprint for heterogeneity-resilient federated learning has been proposed [45]. The treatment of failure, however, remains

largely implicit: canonical hierarchical Federated Averaging assumes that edges remain available and that the cloud is reachable, and surveys of robust federated learning report that defined behaviour beyond restarting training is rarely specified [20]. The third contribution addresses training continuity under edge and cloud failure with explicit, auditable behaviour, and is positioned against this work in Chapter 5.

2.5 Open Challenges

The survey above identifies three gaps between hierarchical federated learning as described in the literature and as it must operate on the edge-cloud continuum. Each gap corresponds to one contribution of this thesis and is referred to from the corresponding chapter.

Aggregation under double non-IID data. Optimisation-based and personalised methods operate on a flat topology; clustered methods rely on discrete membership and parameter-space similarity; hierarchical methods either redistribute a single model or fix the degree of personalisation by a manual schedule, and assume reliable, full edge participation. No existing method produces a distinct model per edge from functional similarity, while adapting the sharing–specialisation balance to observed training dynamics under partial participation. This gap is addressed in Chapter 3.

Participant selection for efficiency under a hierarchical partition. Existing selection strategies improve efficiency in flat federated learning but act at a single tier and assume a static client-to-edge assignment. No existing method selects participants for training efficiency in a way that exploits the two-level hierarchical structure and adapts the selection during training rather than fixing it in advance. This gap is addressed in Chapter 4.

Resilient orchestration under failure. Hierarchical federated learning concentrates on algorithmic behaviour under normal operation and leaves failure handling implicit, with no defined behaviour beyond restart and no record of coordination decisions. No existing method maintains training continuity under repeated edge and cloud failure while providing an auditable account of the actions taken. This gap is addressed in Chapter 5.

2.6 Summary

This chapter has characterised the heterogeneity that makes federated learning difficult on the edge-cloud continuum and has surveyed the developments in aggregation, participant selection, and orchestration that the remainder of the thesis builds upon. Statistical heterogeneity, its two-tier form in the hierarchical setting, system heterogeneity, and partial participation were formalised once here so that the contribution chapters can refer to them without restatement. The survey showed that existing aggregation methods either operate on a flat topology, rely on discrete parameter-space clustering, or fix the degree of personalisation by a manual rule; that existing selection methods improve efficiency but do not exploit the hierarchical structure or adapt during training; and that orchestration work has addressed resources and communication but not training continuity under failure. These three

gaps define the contributions that follow. The first, a framework for personalised aggregation under double non-IID data, is developed in Chapter 3.

Chapter 3

Fedge: A Bayesian Non-Parametric Hierarchical Federated Learning Framework

This chapter proposes Fedge, a hierarchical federated learning (HFL) framework in which the cloud server maintains a variable family of global models rather than a single shared model. The number of models in the family, the assignment of edges to models, and the aggregation operation at each tier are inferred jointly during training under a Bayesian non-parametric prior. The contribution of this chapter is twofold. The first is the framework itself, which subsumes the canonical single-model HFL of HierFAVG [13] and the per-edge personalised HFL of PHE-FL [37] and ESPerHFL [35] as boundary cases. The second is the Fedge algorithm, an inference-and-optimisation procedure that drives the variable family using two-level heterogeneity signals and produces a soft assignment between edges and cloud-tier models. The framework is positioned by an explicit three-way trade-off between personalisation, convergence, and participation robustness, informally referred to as the personalisation–convergence–robustness (P–C–R) trilemma, and is exposed to the operator through a single sensitivity parameter that traces a continuous path through the trade-off frontier. To the best of the author’s knowledge, the combination of Dirichlet Process inference over a cloud-tier model family, soft edge-to-model membership, two-level heterogeneity signals at the client–edge and edge–cloud tiers, and explicit handling of partial participation has not previously been studied in the three-tier hierarchical federated learning setting.

3.1 System Model and the Three-Way Trade-off

This section establishes the system model used throughout the chapter and introduces the analytical lens through which the contribution is positioned. The setting is the standard three-tier client–edge–cloud hierarchy of [13], extended with an explicit two-level heterogeneity model and a partial-participation assumption that matches realistic edge-IoT deployments.

3.1.1 Three-Tier Hierarchical Setting

Let $\mathcal{E} = \{1, 2, \dots, M\}$ denote the set of edge servers and $\mathcal{C}_e = \{c_{e,1}, \dots, c_{e,|\mathcal{C}_e|}\}$ the set of clients served by edge e . Edge sets are disjoint, so the total number of clients is $N = \sum_{e=1}^M |\mathcal{C}_e|$. Each client i holds a private local dataset $\mathcal{D}_i$ of size D_i . A single cloud server orchestrates the topmost aggregation step. Training proceeds over T cloud rounds. Within each cloud round, each participating edge runs E edge aggregation

periods, and within each edge aggregation period, each participating client performs τ local stochastic gradient descent (SGD) steps. The aggregation schedule is fixed in this chapter; adaptive scheduling is addressed in Chapter 4.

3.1.2 Two-Level Non-IID Heterogeneity Model

Data heterogeneity in HFL operates at two levels simultaneously, as observed by [37] and quantified formally by [34]. The first level captures heterogeneity between edges: the marginal label distribution at one edge differs from that at another. The second level captures heterogeneity within an edge: the marginal label distributions of the clients under the same edge differ from each other.

Both levels are parameterised by independent Dirichlet partitions following the protocol of [18]. The global dataset is first partitioned across the M edges using $\text{Dir}(\alpha_{\text{edge}})$, producing inter-edge skew controlled by α_{edge} . Within each edge e , the data assigned to that edge is then partitioned across $\mathcal{C}_e$ using $\text{Dir}(\alpha_{\text{client}})$, producing intra-edge skew controlled by α_{client} . Small values of either parameter produce strong skew at the corresponding level.

Two scalar quantities appear in standard HFL convergence bounds. The intra-edge gradient dissimilarity at edge e is defined as

$$\zeta_{\text{intra}}^2(e) = \frac{1}{|\mathcal{C}_e|} \sum_{i \in \mathcal{C}_e} \|\nabla F_i(w) - \nabla F_e(w)\|^2, \quad (3.1)$$

and the inter-edge gradient dissimilarity is defined as

$$\zeta_{\text{inter}}^2 = \frac{1}{M} \sum_{e=1}^M \|\nabla F_e(w) - \nabla F(w)\|^2, \quad (3.2)$$

where F_i , F_e , and F denote the local, edge, and global loss respectively. A standard HierFAVG-style convergence bound contains the sum of these two terms, and any algorithm that aims to tighten the bound under non-IID data must address both levels [34, 46].

3.1.3 Partial Participation Model

At cloud round t , only a subset of clients at each edge participate. Let $\mathcal{S}_e^t \subseteq \mathcal{C}_e$ denote the set of participating clients at edge e in round t , with per-edge participation rate $\rho_e^t = |\mathcal{S}_e^t|/|\mathcal{C}_e|$. The participation rate is permitted to vary across edges and across rounds, reflecting the intermittent device availability that characterises edge-IoT deployments [9]. Full participation, $\rho_e^t = 1$ for all e and t , is treated as a special case rather than as the default. Most personalisation-focused HFL works [35, 37] are evaluated under full participation only; one design requirement of the framework introduced in this chapter is that the algorithm and its analysis remain valid as ρ_e^t decreases.

3.1.4 The Three-Way Trade-off between Personalisation, Convergence, and Robustness

In the three-tier HFL setting under two-level non-IID data and partial participation, three competing objectives arise. Each can be optimised in isolation, but no aggregation rule can maximise all three simultaneously. This subsection names the three objectives and characterises the tension between each pair. The trade-off is presented

qualitatively here; the algorithm developed in Sections 3.2 and 3.3 is designed to expose a single parameter that allows an operator to position the system at any point of the trade-off frontier.

The three objectives. The first objective is *personalisation depth* (P), defined as the average pairwise distance between cloud-tier output models. A scheme that produces one global model has $P = 0$. A scheme that produces a distinct model per edge has the highest value of P . The second objective is *convergence guarantee tightness* (C), defined as the inverse of the dependence of the convergence bound on the two-level heterogeneity terms ζ_{intra}^2 and ζ_{inter}^2 . A heterogeneity-immune bound, as derived by [34], provides the maximum value of C . The third objective is *participation robustness* (R), defined as the inverse of the degradation in final-round accuracy as ρ_e^t decreases from full participation to a deployment-relevant minimum.

Pairwise tensions. Each pair of objectives admits a structural tension, illustrated in Figure 3.1. Stronger personalisation increases the effective heterogeneity that each per-cluster model encounters during aggregation, which loosens the convergence bound; this is the P -versus- C tension. Tight convergence under non-IID data is achieved by drift-correction mechanisms such as control variates or proximal terms; the original SCAFFOLD analysis [22] proved robustness to client sampling under the assumption of unbiased, uniformly distributed participation. Subsequent work has shown that this assumption fails in realistic deployments: under cyclic or non-uniform participation, control variates become stale and biased between rounds, raising the communication-round complexity from $\mathcal{O}(\epsilon^{-2})$ to $\mathcal{O}(\kappa^2 \epsilon^{-4})$, where κ denotes the data heterogeneity [47]. A separate analysis has further shown that the SCAFFOLD bias does not decrease with the number of clients, breaking the linear-speedup property that motivates the algorithm [48]. This is the C -versus- R tension: the mechanisms required to tighten the convergence bound become themselves a source of bias once participation is intermittent. Strong personalisation requires each per-edge model to be trained on enough signal, and when the cloud maintains multiple distinct models, each model receives on average only a fraction of the available participating clients per round; lower participation rates therefore amplify the variance of each model. This last is the P -versus- R tension.

Boundary cases. Each canonical HFL approach occupies a corner of the three-way trade-off, with a specific limitation that prevents it from occupying the interior. HierFAVG [13] and MTGC [34] produce a single global model and therefore have $P = 0$. MTGC achieves a heterogeneity-immune convergence bound, but requires per-client and per-edge control variates to be stored and updated statefully across rounds, an assumption that conflicts with the cross-device federated learning setting in which clients are stateless and may not participate consecutively [9]. PHE-FL [37] and ES-PerHFL [35] produce one model per edge and therefore have maximal P ; both are evaluated under full participation only, and both fix the cluster count at $K = M$ by construction, providing no mechanism for two near-identical edges to share a model. CHPFL [49] and CFLHKD [50] occupy points between the two corners through clustering: CHPFL uses K-Means++ with K supplied as a hyperparameter, and CFLHKD adopts bi-level aggregation between cluster and cloud with K similarly fixed by the operator and clustering driven by heuristic information-theoretic metrics. HPFL [51]

Figure 3.1 placeholder. Three-way trade-off triangle (see CAT . jpg, with labels updated as described in the chapter notes).

FIGURE 3.1: The three-way trade-off between personalisation, convergence, and robustness. Each vertex represents a single objective; each edge represents the tension between two objectives. No HFL aggregation rule can simultaneously occupy all three vertices.

addresses the three-tier setting but with a fixed clustering structure determined offline and a semi-asynchronous schedule optimised for latency rather than for two-level heterogeneity. None of these methods infers the cluster count from data during training, and none uses two-level heterogeneity signals to drive split and merge decisions.

3.1.5 Output Specification

A training algorithm in this setting produces, at the end of round T , a set of cloud-tier models $\{w_1^T, w_2^T, \dots, w_{K^T}^T\}$, where K^T is the number of distinct models maintained. Three regimes are distinguished. The *global regime* has $K = 1$ throughout training and is the regime of HierFAVG, HierMo [52], MTGC, and the partial-participation analysis of [46]. The *fully personalised regime* has $K = M$ throughout training and is the regime of PHE-FL and ESPerHFL. The *variable regime*, in which $1 \leq K^t \leq M$ is determined adaptively during training, is the regime targeted by the present chapter. No prior HFL work occupies this regime with a Bayesian non-parametric formulation, soft edge-to-model assignment, and explicit handling of partial participation.

3.2 The Fedge Framework

This section introduces the Fedge framework. The framework is a formulation rather than an algorithm: it specifies what the cloud tier maintains, what the aggregation operation produces, and what stochastic structure governs the family of cloud-tier models. The algorithm that realises this framework is presented in Section 3.3.

3.2.1 The Variable Cloud-Tier Model Family

At cloud round t , the cloud server maintains a finite collection of global models

$$\mathcal{W}^t = \{w_1^t, w_2^t, \dots, w_{K^t}^t\}, \quad (3.3)$$

where the cardinality K^t is a random variable inferred from the data rather than a hyperparameter. Each w_k^t is a complete model of the same dimensionality as a single HierFAVG global model; the framework does not address model heterogeneity across k . The collection $\mathcal{W}^t$ is referred to throughout the chapter as the *Fedge family*. The framework name reflects this: Fedge denotes the family of cloud-tier global models that coexist, are created, and are merged in response to changes in the distribution of data across edges.

The architecture is illustrated in Figure 3.2. Edge servers receive blended models from the cloud, perform edge-tier aggregation across their participating clients, and return updated models to the cloud, which then revises both the family $\mathcal{W}^t$ and the assignment of edges to family members.

Figure 3.2 placeholder. Three-tier Fedge architecture (see `fedgearchitecture.jpg`, redrawn to show K^t cloud-tier models with Π^t blending arrows to edges).

FIGURE 3.2: The three-tier Fedge architecture. The cloud maintains the variable family $\mathcal{W}^t$ of K^t global models. Each edge e receives a blended model $\tilde{w}_e^t = \sum_k \Pi_{e,k}^t w_k^t$ governed by the soft membership matrix Π^t . Clients train locally using SGD and return updates to the edge.

3.2.2 Dirichlet Process Prior over the Family

The variability of K^t is governed by a Dirichlet Process prior over the parameters of the family members, following the standard non-parametric clustering construction of [53] and its federated adaptation in flat topology by [54]. The generative model at round t is

$$G \sim \text{DP}(\alpha_0(\lambda), G_0), \quad (3.4)$$

$$\theta_e \sim G, \quad e = 1, \dots, M, \quad (3.5)$$

$$w_k^t \mid \theta_e = \theta_k \sim p(w_k^t \mid \theta_k), \quad (3.6)$$

where G is a discrete random measure drawn from a Dirichlet Process with concentration parameter $\alpha_0(\lambda)$ and base measure G_0 , and θ_e is the latent cluster parameter for edge e . The Dirichlet Process induces a partition of the M edges into $K^t \leq M$ distinct clusters, with K^t a posterior quantity inferred from data rather than fixed in advance. The number of clusters under the Dirichlet Process prior follows the stick-breaking process and is observed empirically to stabilise within a small number of rounds, a property exploited in the convergence analysis of Section 3.4.2.

The concentration parameter α_0 is not exposed to the operator directly. Instead, the framework defines a single user-facing sensitivity parameter $\lambda \in [0, 1]$ and maps it monotonically to α_0 such that $\lambda = 0$ recovers $\alpha_0 \rightarrow 0$ (the partition concentrates on a single cluster) and $\lambda = 1$ recovers $\alpha_0 \rightarrow \infty$ (the partition is permitted to grow up to M distinct clusters). The mapping is specified in Section 3.3.4. The role of λ is to expose the three-way trade-off to the operator: $\lambda = 0$ recovers the convergence corner ($K = 1$), $\lambda = 1$ recovers the personalisation corner ($K = M$), and intermediate values position the system in the interior of the trade-off frontier.

3.2.3 Soft Edge-to-Model Membership

Each edge e is associated at round t with a probability vector over the family members,

$$\Pi_{e,\cdot}^t = (\Pi_{e,1}^t, \Pi_{e,2}^t, \dots, \Pi_{e,K^t}^t), \quad \sum_{k=1}^{K^t} \Pi_{e,k}^t = 1, \quad \Pi_{e,k}^t \geq 0. \quad (3.7)$$

The full $M \times K^t$ matrix Π^t is the *soft membership matrix* and is the central data structure of the framework. The term *soft membership* is used here in its standard sense from mixture-model and federated-clustering literature [33, 55]: each edge holds a probability of belonging to each cluster, rather than being assigned to exactly one cluster. The soft formulation has been studied in flat, single-tier federated learning

by Ruan and Joe-Wong [33] and by Wu, Imbiriba, and Closas [55]; the latter develops a Bayesian framework for the client-to-cluster association problem on a single-server topology, with experimental validation on label-skew partitions of Digits-Five, Fashion-MNIST, and CIFAR-10 at scales of approximately ten clients per cluster. Neither work addresses the two-level heterogeneity signal structure of the three-tier hierarchy, the edge-tier aggregation step, or the partial-participation setting that motivates Fedge. The complementary case, *hard membership*, in which each edge is assigned to exactly one cluster, corresponds to the degenerate case in which every row of Π^t has a single non-zero entry equal to one; this is the case in CFL [31] and IFCA [32].

Two consequences follow from the soft formulation. First, an edge whose distribution lies between two clusters is not forced to commit to one, so the framework degrades gracefully when an edge sits near a cluster boundary. Second, when the participating clients at edge e become scarce in round t due to partial participation, the membership distribution redistributes weight to neighbouring clusters that still receive sufficient signal, providing robustness without requiring an explicit fallback rule.

The model delivered to edge e at the start of round $t + 1$ is the membership-weighted blend

$$\tilde{w}_e^{t+1} = \sum_{k=1}^{K^t} \Pi_{e,k}^t w_k^t. \quad (3.8)$$

Local training at edge e proceeds from $\tilde{w}_e^{t+1}$, and the resulting update is used to revise both the family $\mathcal{W}$ and the membership matrix Π , as detailed in Section 3.3.

3.2.4 Framework Properties

Three structural properties of the framework are stated explicitly before the algorithm is introduced.

Subsumption of canonical HFL. At $\lambda = 0$, the Dirichlet Process prior degenerates and the inference procedure produces $K^t = 1$ for all t , with $\Pi_{e,1}^t = 1$ for every edge. The blend reduces to a single global model, and the framework reduces to HierFAVG, or to MTGC if drift-correction terms are added. At $\lambda = 1$, the Dirichlet Process prior favours the maximum partition and produces $K^t = M$ for all t , with $\Pi^t = I_M$ (the identity matrix). The framework reduces to a per-edge personalised scheme of the PHE-FL or ESPerHFL family.

Two-level signal coupling. The intra-edge term $\zeta_{\text{intra}}^2(e)$ from Equation (3.1) and the inter-edge term ζ_{inter}^2 from Equation (3.2) are both available at the cloud as measurable quantities derived from the edge updates. The inference procedure of Section 3.3 uses both signals jointly to drive the posterior over K^t and Π^t . No prior elastic- K method, including [54] and [56], uses two-level signals, because both operate on flat topologies.

Partial-participation compatibility. The Dirichlet Process posterior is updated from the participating subset $\mathcal{S}_e^t$ rather than from $\mathcal{C}_e$, and the soft membership of Equation (3.7) is rescaled accordingly. The convergence analysis of Section 3.4.2 characterises the dependence of the bound on ρ_e^t explicitly.

3.3 The Fedge Algorithm

This section presents the Fedge algorithm, which realises the framework of Section 3.2 as a concrete inference-and-optimisation procedure. The algorithm is reactive: it observes the two-level heterogeneity signals at each cloud round and updates the family $\mathcal{W}^t$ and membership matrix Π^t accordingly.

3.3.1 Two-Level Heterogeneity Signals

At each cloud round t , two scalar signals are computed from the available edge updates. The first is the intra-edge dispersion

$$\hat{\zeta}_{\text{intra}}^2(e, t) = \frac{1}{|\mathcal{S}_e^t|} \sum_{i \in \mathcal{S}_e^t} \|\Delta w_i^t - \Delta w_e^t\|^2, \quad (3.9)$$

where Δw_i^t is the parameter update produced by client i during round t and Δw_e^t is the mean update across the participating clients at edge e . This is the empirical estimator of $\zeta_{\text{intra}}^2(e)$ from Equation (3.1) under partial participation. The second signal is the pairwise inter-edge dissimilarity between edges e and e' ,

$$\hat{d}^2(e, e', t) = \|\Delta w_e^t - \Delta w_{e'}^t\|^2, \quad (3.10)$$

which provides a measurable proxy for the inter-edge component of heterogeneity. Both signals are computed from quantities already available at the cloud after edge-tier aggregation, so no additional communication is required.

The use of two signals rather than one is a structural difference from prior elastic- K methods. DPMM-CFL [54] uses only client-level model distances. FedDAA [56] uses only prototype-based concept-drift signals. Neither method has access to a two-level hierarchy, and neither method's signal can distinguish intra-cluster from inter-cluster divergence. In Fedge, the two signals enter the inference procedure of Section 3.3.2 as complementary inputs: $\hat{\zeta}_{\text{intra}}^2(e, t)$ controls whether the clients of an edge agree internally on cluster assignment, and $\hat{d}^2(e, e', t)$ controls whether two edges should remain in distinct clusters or be merged into one.

3.3.2 Split-Merge Moves

The cardinality K^t of the family $\mathcal{W}^t$ and the membership matrix Π^t are updated each round by a Metropolis–Hastings sampler over the partition structure of the M edges, following the split-merge Markov chain Monte Carlo procedure of [57] and adapted to the federated setting by [54]. Two reversible moves are proposed.

Split move. A split move is proposed when the intra-edge signal of one or more edges exceeds the inter-edge signal of the cluster currently containing those edges. The proposal selects a cluster k and partitions its constituent edges into two subsets according to the leading eigenvector of the pairwise dissimilarity matrix restricted to that cluster. The proposal is accepted with the Metropolis–Hastings probability

$$P_{\text{split}} = \min\left(1, \frac{p(\mathcal{W}^*, \Pi^* \mid \text{data}, \alpha_0(\lambda))}{p(\mathcal{W}^t, \Pi^t \mid \text{data}, \alpha_0(\lambda))} \cdot \frac{q(\mathcal{W}^t, \Pi^t \mid \mathcal{W}^*, \Pi^*)}{q(\mathcal{W}^*, \Pi^* \mid \mathcal{W}^t, \Pi^t)}\right), \quad (3.11)$$

where the asterisked quantities denote the proposed state and q is the proposal distribution induced by the eigenvector partition. If accepted, $K^{t+1} = K^t + 1$ and the membership matrix is updated to reflect the new partition.

Merge move. A merge move is proposed when the pairwise inter-edge dissimilarity between two cluster centroids falls below a posterior-derived threshold that depends on $\alpha_0(\lambda)$ and the cluster sizes. The proposal merges the two clusters and assigns their union the centroid model. Acceptance follows the analogous Metropolis–Hastings rule; if accepted, $K^{t+1} = K^t - 1$.

Both moves are reversible, so the sampler maintains detailed balance with respect to the Dirichlet Process posterior. The cardinality K^t is observed to stabilise within a small number of rounds, after which neither move is accepted; this empirical behaviour is documented for the flat case by [54] and is exploited in the convergence analysis of Section 3.4.2.

3.3.3 Aggregation with Soft Membership

Once K^t and Π^t are updated, the family $\mathcal{W}^{t+1}$ is produced by membership-weighted aggregation. For each cluster k , the new global model is

$$w_k^{t+1} = \frac{\sum_{e=1}^M \Pi_{e,k}^t n_e^t w_e^{t+1}}{\sum_{e=1}^M \Pi_{e,k}^t n_e^t}, \quad (3.12)$$

where w_e^{t+1} is the model returned by edge e after its edge-tier aggregation in round t , and $n_e^t = |\mathcal{S}_e^t|$ is the number of participating clients at edge e in round t . The denominator is the cluster’s effective sample size at round t . An edge contributes to the update of a cluster in proportion to its membership and its participating sample count; an edge with low membership in cluster k contributes weakly, and an edge with no participating clients in round t contributes zero.

This is a generalisation of the HierFAVG aggregation [13]: setting $K^t = 1$ and $\Pi_{e,1}^t = 1$ recovers the standard HierFAVG update. It is also a generalisation of hard-membership cluster aggregation [31, 32]: setting Π^t to a permutation matrix recovers hard cluster assignment. The soft formulation is what permits an edge to contribute to multiple clusters in proportion to its membership posterior, providing the graceful degradation property of Section 3.2.3.

3.3.4 The Sensitivity Parameter

The single operator-facing parameter $\lambda \in [0, 1]$ controls the sensitivity of the algorithm to divergence in heterogeneity. Internally, λ enters the Dirichlet Process concentration through the monotone mapping

$$\alpha_0(\lambda) = \alpha_{\min} \cdot \left(\frac{\alpha_{\max}}{\alpha_{\min}} \right)^\lambda, \quad (3.13)$$

where $\alpha_{\min}$ and $\alpha_{\max}$ are fixed framework constants chosen so that $\lambda = 0$ produces $K^t \rightarrow 1$ and $\lambda = 1$ produces $K^t \rightarrow M$ on representative non-IID partitions. The operator perceives λ as a sensitivity dial: a small value of λ favours fewer global models with stronger pooling across edges, and a large value of λ favours more global models with stronger specialisation per edge.

Algorithm 2 Fedge

Require: Edges $\mathcal{E}$, sensitivity parameter $\lambda \in [0, 1]$, total cloud rounds T , edge aggregation periods E , local SGD steps τ , initial model w_0

Ensure: Final family $\mathcal{W}^T$ and membership matrix Π^T

- 1: Initialise $K^0 \leftarrow 1$, $\mathcal{W}^0 \leftarrow \{w_0\}$, $\Pi^0 \leftarrow \mathbf{1}_{M \times 1}$
- 2: Set $\alpha_0(\lambda)$ from Equation (3.13)
- 3: **for** $t = 0, 1, \dots, T - 1$ **do**
- 4: **for** each edge $e \in \mathcal{E}$ **in parallel do**
- 5: Compute blended model $\tilde{w}_e^t \leftarrow \sum_{k=1}^{K^t} \Pi_{e,k}^t w_k^t$
- 6: Sample participating clients $\mathcal{S}_e^t \subseteq \mathcal{C}_e$
- 7: **for** each edge aggregation period $1, \dots, E$ **do**
- 8: **for** each client $i \in \mathcal{S}_e^t$ **in parallel do**
- 9: Perform τ steps of local SGD from current edge model
- 10: **end for**
- 11: Edge aggregates client updates and produces w_e^{t+1}
- 12: **end for**
- 13: Return w_e^{t+1} and per-client updates $\{\Delta w_i^t\}_{i \in \mathcal{S}_e^t}$ to the cloud
- 14: **end for**
- 15: Compute intra-edge signals $\hat{\zeta}_{\text{intra}}^2(e, t)$ from Equation (3.9) for all e
- 16: Compute pairwise inter-edge signals $\hat{d}^2(e, e', t)$ from Equation (3.10) for all e, e'
- 17: Propose split-merge moves and accept according to Equation (3.11)
- 18: Update partition cardinality K^{t+1} and membership matrix Π^{t+1}
- 19: **for** each cluster $k = 1, \dots, K^{t+1}$ **do**
- 20: Compute w_k^{t+1} by membership-weighted aggregation, Equation (3.12)
- 21: **end for**
- 22: Set $\mathcal{W}^{t+1} \leftarrow \{w_1^{t+1}, \dots, w_{K^{t+1}}^{t+1}\}$
- 23: **end for**
- 24: **return** $\mathcal{W}^T, \Pi^T$

The behaviour of the algorithm along the three-way trade-off frontier is determined by λ . At $\lambda = 0$, the algorithm recovers MTGC-style single-model HFL, with the strongest convergence guarantee but no personalisation. At $\lambda = 1$, the algorithm recovers per-edge personalised HFL of the PHE-FL or ESPerHFL family, with full personalisation but weaker convergence and reduced robustness under low participation. Intermediate values of λ produce intermediate values of K^t chosen by the inference procedure to reflect the actual distributional structure of the federation.

3.3.5 Algorithm Pseudocode

The Fedge algorithm is summarised as Algorithm 2. The procedure is invoked at the start of each cloud round and produces the family $\mathcal{W}^{t+1}$ and the membership matrix Π^{t+1} to be used in the next round.

3.3.6 Computational and Communication Cost

This subsection formalises the per-round and total training cost of the Fedge algorithm and compares it against the canonical HFL baselines reviewed in Section 3.1.4.

Notation. Let $|w|$ denote the number of parameters of a single model and b the number of bits per parameter (typically $b = 32$ for single-precision floating point). For simplicity of exposition, the comparison assumes a uniform number of clients per edge $|\mathcal{C}_e| = C$ and a uniform participation rate $\rho_e^t = \rho$; the analysis generalises directly to non-uniform settings by substituting the corresponding sums. One cloud round consists of E edge aggregation periods, and each edge aggregation period consists of one client–edge exchange in which each participating client uploads its update to its edge server and the edge sends the aggregated model back.

Per-round upload cost. The per-round upload cost of HierFAVG is

$$U_{\text{HierFAVG}}^t = b \cdot |w| \cdot M \cdot (1 + E\rho C), \quad (3.14)$$

accounting for $E\rho C$ client uploads per edge across the E edge aggregation periods plus one edge upload to the cloud, summed across the M edges. The per-round upload cost of Fedge is

$$U_{\text{Fedge}}^t = b \cdot |w| \cdot M \cdot (1 + E\rho C) + b \cdot M, \quad (3.15)$$

where the additive term $b \cdot M$ accounts for the transmission of one intra-edge dispersion scalar per edge from Equation (3.9). Since $|w| \gg 1$ for any modern neural network, the additive term is negligible and

$$U_{\text{Fedge}}^t = U_{\text{HierFAVG}}^t \cdot \left(1 + \frac{1}{|w| (1 + E\rho C)}\right) \approx U_{\text{HierFAVG}}^t. \quad (3.16)$$

By contrast, the worst-case per-round upload cost of MTGC is

$$U_{\text{MTGC}}^t = 2 \cdot b \cdot |w| \cdot M \cdot (1 + E\rho C) = 2 \cdot U_{\text{HierFAVG}}^t, \quad (3.17)$$

because each client–edge and edge–cloud transmission carries a control variate of equal dimensionality to the model in the worst case. Fang *et al.* argue that this overhead can be reduced in practice by estimating control variates from model differences rather than transmitting them explicitly [34], but the dimensionality of the control state remains a function of the model size, and the controls require persistent per-client and per-edge storage across rounds. Fedge avoids the question of control-variate overhead entirely: the inter-cluster heterogeneity reduction is achieved through the soft membership matrix Π^t and the model family $\mathcal{W}^t$, both of which are stored at the cloud and never transmitted. The per-round upload cost of ESPerHFL [35] and PHE-FL [37] matches that of HierFAVG at the edge–cloud step, because each edge transmits a single aggregated model to the cloud; however, ESPerHFL additionally exchanges per-client personalisation parameters (the local model, the mixing weight, and normalisation parameters) at the client–edge step in every iteration, inflating the client–edge cost above that of HierFAVG.

Per-round download cost. The per-round download cost of HierFAVG, ESPerHFL, and Fedge is

$$D_{\text{HierFAVG}}^t = D_{\text{ESPerHFL}}^t = D_{\text{Fedge}}^t = b \cdot |w| \cdot M \cdot (1 + E\rho C), \quad (3.18)$$

with the blended model $\tilde{w}_e^t$ delivered to each edge in Fedge having the same dimensionality as a single model, since the blend of Equation (3.8) is computed at the cloud

before transmission. PHE-FL incurs an additional download per cloud round for the complement model:

$$D_{\text{PHE-FL}}^t = b \cdot |w| \cdot M \cdot (2 + E\rho C). \quad (3.19)$$

MTGC doubles the download cost for the same reason as upload:

$$D_{\text{MTGC}}^t = 2 \cdot b \cdot |w| \cdot M \cdot (1 + E\rho C). \quad (3.20)$$

Per-round cloud computational cost. The per-round cloud-side computational cost of HierFAVG, MTGC, ESPerHFL, and PHE-FL is dominated by the weighted aggregation of M models, which is $O(M \cdot |w|)$ floating-point operations. The per-round cloud-side computational cost of Fedge has three components: the inter-edge signal computation in $O(M^2 \cdot |w|)$, the split-merge proposal in $O(M^2 \cdot |w|)$ dominated by the eigenvector computation in a split move, and the membership-weighted aggregation in $O(K^t \cdot M \cdot |w|)$. Since $K^t \leq M$, the total reduces to

$$\mathcal{F}_{\text{Fedge}}^t = O(M^2 \cdot |w|). \quad (3.21)$$

The overhead factor relative to HierFAVG is M , which is meaningful but bounded by the number of edges; for the typical deployment scale of $M \leq 20$ assumed in this chapter, the absolute cost remains modest compared with the cost of model transmission.

Total training cost. The total bits transmitted over T cloud rounds to reach a target accuracy ϵ is

$$\mathcal{B}(\epsilon) = T(\epsilon) \cdot (U^t + D^t), \quad (3.22)$$

where $T(\epsilon)$ is the number of cloud rounds required by the algorithm to reach ϵ . The dependence of $T(\epsilon)$ on the system parameters is established by the convergence analysis presented later in this chapter; for fixed T , the per-algorithm upload and download expressions of Equations (3.15) to (3.20) provide the per-round multiplier.

Comparison summary. Table 3.1 consolidates the per-round upload, download, cloud computational cost, and the relative total-bits multiplier against HierFAVG for the five algorithms considered. The Fedge algorithm preserves the communication cost of HierFAVG up to the negligible scalar overhead of Equation (3.16), avoids the per-client control-variate storage required by MTGC and the per-client personalisation parameters exchanged by ESPerHFL, and incurs an M -fold cloud computational overhead relative to HierFAVG arising from the split-merge inference procedure. The cloud computational overhead is bounded by the number of edges M rather than by the number of clients N , which is the relevant property for cross-device deployments in which $N \gg M$.

3.4 Convergence Analysis

This section establishes the convergence guarantee of the Fedge algorithm. The analysis is conducted under standard non-convex federated learning assumptions extended with two additional assumptions specific to the framework: bounded two-level heterogeneity and stabilisation of the cluster count. The main result, Theorem 3.1, bounds the average squared gradient norm of the per-cluster objective after a warm-up period during which the inference procedure of Section 3.3.2 stabilises.

TABLE 3.1: Per-round cost comparison for the Fedge algorithm against four hierarchical federated learning baselines. Notation as in Section 3.3.6. The relative total-bits multiplier is the ratio of total transmitted bits relative to HierFAVG under the same number of cloud rounds T . The MTGC upload and download figures are worst-case; Fang *et al.* [34] note that control variates can be estimated from model differences with reduced transmission cost at the price of persistent per-client and per-edge storage. The ESPerHFL row reports edge–cloud cost; client–edge cost is higher due to per-client personalisation parameters.

Algorithm	Per-round upload (bits)	Per-round download (bits)	Cloud compute (flops)	Total-bits vs HierFAVG
HierFAVG [13]	$b w M(1 + E\rho C)$	$b w M(1 + E\rho C)$	$O(M w)$	1
MTGC [34]	$2b w M(1 + E\rho C)$	$2b w M(1 + E\rho C)$	$O(M w)$	2
ESPerHFL [35]	$b w M(1 + E\rho C)$	$b w M(1 + E\rho C)$	$O(M w)$	1
PHE-FL [37]	$b w M(1 + E\rho C)$	$b w M(2 + E\rho C)$	$O(M w)$	$1 + \frac{1}{2(1+E\rho C)}$
Fedge	$b w M(1 + E\rho C) + bM$	$b w M(1 + E\rho C)$	$O(M^2 w)$	≈ 1

The bound exposes three terms with distinct interpretations and recovers the canonical single-model and per-edge personalised bounds as boundary cases at $\lambda = 0$ and $\lambda = 1$ respectively. A proof sketch is provided in Section 3.4.4; the full proof is deferred to Appendix A.

3.4.1 Assumptions

Five assumptions are required for the analysis. The first three are standard in the non-convex federated learning literature [22, 34, 46]. The fourth and fifth are specific to the Fedge framework.

A1 (L -smoothness). Each local loss function F_i is differentiable and its gradient is L -Lipschitz continuous:

$$\|\nabla F_i(w) - \nabla F_i(w')\| \leq L \|w - w'\|, \quad \forall w, w' \in \mathbb{R}^{|w|}, \quad \forall i. \quad (3.23)$$

A2 (Bounded stochastic-gradient variance). The stochastic gradient computed by client i on a mini-batch ξ is an unbiased estimator of ∇F_i , with variance bounded uniformly by σ^2 :

$$\mathbb{E}_{\xi} [\nabla F_i(w; \xi)] = \nabla F_i(w), \quad \mathbb{E}_{\xi} \|\nabla F_i(w; \xi) - \nabla F_i(w)\|^2 \leq \sigma^2. \quad (3.24)$$

A3 (Bounded two-level heterogeneity). The intra-edge and inter-edge gradient dissimilarity terms of Equations (3.1) and (3.2) are bounded uniformly in w :

$$\zeta_{\text{intra}}^2(e) \leq G_{\text{intra}}^2, \quad \zeta_{\text{inter}}^2 \leq G_{\text{inter}}^2, \quad \forall e, \quad \forall w. \quad (3.25)$$

A4 (Cluster-quality factor). After cluster stabilisation, the average per-cluster heterogeneity is at most a factor γ of the inter-edge heterogeneity:

$$\frac{1}{K^*} \sum_{k=1}^{K^*} \zeta_k^2 \leq \gamma G_{\text{inter}}^2, \quad \gamma \in (0, 1], \quad (3.26)$$

where ζ_k^2 is the gradient dissimilarity between edges sharing membership in cluster k , and γ is the cluster-quality factor. The assumption states that clustering reduces effective inter-edge heterogeneity by a factor of at least γ . The factor γ depends on the partition produced by the inference procedure: a well-separated partition yields a small γ , and an unstructured federation yields γ close to one.

A5 (Stabilisation of K^t). There exists a warm-up round $T_0 \in \mathbb{N}$ such that the cluster count is constant at a value K^* for all subsequent rounds:

$$K^t = K^*, \quad \forall t \geq T_0. \quad (3.27)$$

The assumption is justified empirically by the stabilisation behaviour of split-merge Markov chain Monte Carlo inference under Dirichlet Process priors reported by [54] in the flat clustered federated learning setting; the same behaviour is observed in the experiments of Section 3.2, where T_0 is measured directly.

3.4.2 Stabilisation Window

The convergence theorem of Section 3.4.3 bounds the optimisation error from round T_0 onward rather than from round zero. The treatment of the warm-up window is consistent with the practice in clustered federated learning analyses [31, 54], where the partition inference is treated as a separate phase from the per-cluster optimisation. During the warm-up, the family $\mathcal{W}^t$ is shaped by the split-merge proposals of Section 3.3.2, and the soft membership matrix Π^t is updated each round. After T_0 , the partition is fixed and the optimisation reduces to K^* parallel per-cluster federated learning problems coupled only through the soft membership weights of edges that lie between clusters.

The value of T_0 is not bounded analytically in this chapter, as such a bound requires assumptions on the geometry of the parameter space at initialisation that cannot be verified for arbitrary neural-network architectures. The empirical value of T_0 is reported in Section ??.

3.4.3 Main Convergence Theorem

The main theoretical result of the chapter is stated as follows.

Theorem 3.1 (Fedge convergence). *Under Assumptions A1–A5, let the local step size satisfy $\eta = \mathcal{O}\left(1/(L\sqrt{(T-T_0)\bar{\rho}C})\right)$, where $\bar{\rho} = \frac{1}{M} \sum_{e=1}^M \rho_e$ is the average per-edge participation rate. Then the iterates of the Fedge algorithm satisfy*

$$\frac{1}{T-T_0} \sum_{t=T_0}^{T-1} \frac{1}{K^*} \sum_{k=1}^{K^*} \mathbb{E} \|\nabla F_k(w_k^t)\|^2 \leq \underbrace{\mathcal{O}\left(\frac{L\Delta_0 + \sigma^2}{\sqrt{(T-T_0)\bar{\rho}C}}\right)}_{(i) \text{ optimisation noise}} + \underbrace{\mathcal{O}\left(\frac{\gamma G_{\text{inter}}^2}{K^*}\right)}_{(ii) \text{ inter-cluster residual}} + \underbrace{\mathcal{O}\left(\frac{G_{\text{intra}}^2}{\bar{\rho}C}\right)}_{(iii) \text{ intra-edge residual}}, \quad (3.28)$$

where F_k denotes the loss of cluster k and $\Delta_0 = F(w^{T_0}) - F^*$ is the sub-optimality at the end of the warm-up window.

The bound contains three terms, each with a distinct interpretation. Term (i) is the standard non-convex federated learning optimisation rate, decreasing as $1/\sqrt{T - T_0}$ with the total post-warm-up rounds and improving with average participation $\bar{\rho}$ and clients per edge C . Term (ii) is the residual heterogeneity that survives after clustering: it shrinks as the cluster count K^* grows, and is scaled by the cluster-quality factor γ from Assumption A4. Term (iii) is the irreducible intra-edge heterogeneity that clustering cannot reduce, since clusters are formed across edges rather than across clients within an edge.

The structure of the bound makes the role of clustering explicit. Single-model HFL, such as HierFAVG [13] and MTGC [34] in its non-control-variate form, corresponds to $K^* = 1$, in which case term (ii) reduces to $\mathcal{O}(\gamma G_{\text{inter}}^2)$ with no reduction. Per-edge personalised HFL, such as PHE-FL [37] and ESPerHFL [35], corresponds to $K^* = M$, in which case term (ii) is divided by M . Fedge selects K^* from the data, and the bound is minimised when K^* is large enough to drive term (ii) below term (iii) but not so large that term (i) inflates due to small per-cluster sample sizes.

3.4.4 Proof Sketch

The proof of Theorem 3.1 proceeds in four steps. The full derivation is given in Appendix A; this subsection states the four steps and identifies the technical novelty relative to existing HFL convergence proofs.

Step 1: One-step descent inequality. By Assumption A1, L -smoothness of F_k yields the standard descent inequality

$$\mathbb{E}[F_k(w_k^{t+1})] \leq F_k(w_k^t) - \eta \mathbb{E}\langle \nabla F_k(w_k^t), \tilde{g}_k^t \rangle + \frac{L\eta^2}{2} \mathbb{E}\|\tilde{g}_k^t\|^2, \quad (3.29)$$

where $\tilde{g}_k^t$ is the aggregated update applied to cluster k at round t . This step is standard [22].

Step 2: Bound on local-update drift. The drift of the local SGD iterates from the blended model $\tilde{w}_e^t$ is bounded using Assumptions A1, A2, and A3. The drift bound takes the form

$$\mathbb{E}\|w_{i,s}^t - \tilde{w}_e^t\|^2 \leq \mathcal{O}(\eta^2 \tau^2 (\sigma^2 + G_{\text{intra}}^2)), \quad s = 1, \dots, \tau, \quad (3.30)$$

for the local iterates of client $i \in \mathcal{S}_e^t$. This step follows the SCAFFOLD-style analysis of [22] adapted to the hierarchical setting of [34, 46].

Step 3: Variance of the membership-weighted aggregate. This is the step in which the analysis differs structurally from prior HFL proofs. The aggregated update $\tilde{g}_k^t$ for cluster k is a membership-weighted sum of edge updates:

$$\tilde{g}_k^t = \frac{\sum_{e=1}^M \Pi_{e,k}^t n_e^t \Delta w_e^t}{\sum_{e=1}^M \Pi_{e,k}^t n_e^t}. \quad (3.31)$$

The variance of $\tilde{g}_k^t$ decomposes into three contributions: stochastic gradient noise, intra-edge heterogeneity, and inter-edge heterogeneity restricted to the support of

column k of Π^t . After applying Assumption A4 to bound the inter-edge component on the support, the variance bound reads

$$\mathbb{E}\|\tilde{g}_k^t - \nabla F_k(w_k^t)\|^2 \leq \mathcal{O}\left(\frac{\sigma^2}{\bar{\rho}C} + \frac{G_{\text{intra}}^2}{\bar{\rho}C} + \frac{\gamma G_{\text{inter}}^2}{K^*}\right). \quad (3.32)$$

The factor $1/K^*$ in the inter-cluster term emerges from the restriction of Π^t to a single column: under Assumption A4, the average heterogeneity within a cluster is at most $\gamma G_{\text{inter}}^2$, and the bound on the per-cluster variance follows by Jensen's inequality applied to the column-wise sum. The full derivation is given in Appendix A, Section A.3.

Step 4: Telescoping over $T - T_0$ rounds. Substituting Equations (3.30) and (3.32) into Equation (3.29), telescoping the descent inequality from $t = T_0$ to $t = T - 1$, choosing the step size $\eta = \mathcal{O}(1/(L\sqrt{(T - T_0)\bar{\rho}C}))$, and averaging over rounds and clusters yields Equation (3.28). This step is standard once the variance bound of step 3 is established.

The technical contribution of the proof lies in step 3, where the soft membership matrix Π^t enters the variance decomposition. No prior HFL convergence proof treats a soft membership structure, since prior elastic-cluster methods [54, 56] use hard assignment, and prior soft-mixing methods [35, 37] use fixed $K = M$. The bound of Equation (3.32) is therefore new.

3.4.5 Implications for the Three-Way Trade-off

Three corollaries follow directly from Theorem 3.1. Each corresponds to one of the boundary cases identified in Section 3.1.4 and quantifies the position of the corresponding HFL family on the trade-off frontier.

Corollary 3.1 (Convergence corner, $\lambda = 0$). *At $\lambda = 0$, the Fedge algorithm produces $K^* = 1$ almost surely. The bound of Equation (3.28) reduces to*

$$\frac{1}{T - T_0} \sum_{t=T_0}^{T-1} \mathbb{E}\|\nabla F(w^t)\|^2 \leq \mathcal{O}\left(\frac{L\Delta_0 + \sigma^2}{\sqrt{(T - T_0)\bar{\rho}C}}\right) + \mathcal{O}\left(\gamma G_{\text{inter}}^2 + \frac{G_{\text{intra}}^2}{\bar{\rho}C}\right), \quad (3.33)$$

which matches the standard HierFAVG bound of [13] and [46] up to constants. Personalisation is zero, and convergence is governed by the un-reduced two-level heterogeneity.

Corollary 3.2 (Personalisation corner, $\lambda = 1$). *At $\lambda = 1$, the Fedge algorithm produces $K^* = M$ almost surely, with $\Pi^t = I_M$. Term (ii) of Equation (3.28) reduces to $\mathcal{O}(\gamma G_{\text{inter}}^2/M)$, and per-edge participating samples drop to $\rho_e C$ for each cluster. The bound becomes*

$$\frac{1}{T - T_0} \sum_{t=T_0}^{T-1} \frac{1}{M} \sum_{e=1}^M \mathbb{E}\|\nabla F_e(w_e^t)\|^2 \leq \mathcal{O}\left(\frac{L\Delta_0 + \sigma^2}{\sqrt{(T - T_0)\bar{\rho}C}}\right) + \mathcal{O}\left(\frac{\gamma G_{\text{inter}}^2}{M} + \frac{G_{\text{intra}}^2}{\bar{\rho}C}\right). \quad (3.34)$$

The inter-cluster residual is at its minimum but the optimisation noise term receives no benefit from cluster pooling, exposing the personalisation-versus-robustness tension.

Corollary 3.3 (Interior regime, $0 < \lambda < 1$). *For $\lambda \in (0, 1)$, the cluster count K^* takes an intermediate value determined by Assumption A4 and the inference procedure of Section 3.3.2. The bound of Equation (3.28) is minimised at the value of K^* at which the*

marginal reduction in term (ii) equals the marginal cost in terms (i) and (iii). Explicit calculation gives

$$K_{opt}^* = \Theta \left(\frac{\gamma G_{inter}^2 \cdot \bar{\rho} C}{L\Delta_0 + \sigma^2 + G_{intra}^2} \right)^{1/2}, \quad (3.35)$$

up to constants depending on the constants hidden in the $\mathcal{O}$ notation of Equation (3.28).

The interior corollary establishes that the optimal cluster count is not the maximum M but an intermediate value that balances the three terms of the bound. The optimal K^* grows with the inter-edge heterogeneity G_{inter}^2 and with the average participation $\bar{\rho}C$, and shrinks with the optimisation noise and the intra-edge heterogeneity. The result is consistent with the empirical observation that fixed- K clustering methods such as CHPFL [49] require K to be tuned to the data; the Fedge inference procedure selects K^* automatically, with the sensitivity parameter λ providing the operator with control over the trade-off position. The empirical validity of the prediction of Equation (3.35) is examined in Section ??.

Chapter 4

4

Chapter 5

Orchestration for Resilient HFL

HFL deployments in the edge-cloud continuum operate under continuous infrastructure pressure. Edges fail, network links drop, and components at every tier are subject to routine maintenance and unexpected faults. As HFL is moved from controlled experimental settings into production, the orchestration layer, responsible for detecting these events and reassigning workload across surviving components, becomes central to system reliability. It determines whether a transient edge failure stalls a training round, or whether a higher-tier crash forces a full restart.

Recent orchestration work for HFL has made the aggregator hierarchy reactive to runtime events, reconfiguring the training pipeline as failures occur [43–45]. These approaches share four properties that limit them in deployments at scale. Orchestration decisions are issued by a single logical controller at the cloud tier, which is a single point of failure and a latency floor: decision latency is bounded by the round-trip to the controller, and orchestration halts entirely if the controller itself fails. The decision logic is fixed at design time as a rule table, so failure modes outside the rules cannot be addressed and new patterns cannot be accommodated. No explanation accompanies any committed decision; an operator inspecting a logged reassignment sees only the action, not the reasoning that produced it. Higher-tier failover, in which an aggregator at the edge tier or above itself crashes, is not addressed: existing reactive designs focus on client-tier dynamics and either fall back to a system restart or leave the case undefined. This chapter addresses the third research objective of this thesis, set out in Section 1.4 of Chapter 1: the development of a new orchestration design for HFL that maintains training continuity under edge and cloud failures on the edge-cloud continuum, with explicit failure handling and auditable decisions.

This chapter develops two orchestration designs that address these limitations within a single architectural template. The template places orchestration in a set of cooperating nodes at the cloud tier (referred to as Fedge nodes) rather than in a single logical controller, so the system has no centralised point of failure. Both designs act proactively: they prepare a corrective action during the warning window that precedes a failure and commit it on the transition to the failed state. The two designs differ in how the corrective action is produced. The first, the Rule-based Fedge, is deterministic and precomputes the action through a four-module control loop. The second, the Agentic Fedge, is deliberative and produces the action through a three-tier design in which a deterministic triage and constraint validator forms the base, a Large Language Model (LLM) deliberative path produces structured actions with natural-language rationales, and an episodic memory store accumulates compact summaries of past episodes. Both designs cover two failure classes: edge-level failures within a single Fedge partition, and cross-partition failover when a Fedge itself crashes.

The contributions of this chapter are as follows.

- A formulation of HFL orchestration as a distributed control problem, with a unified failure model that covers both within-partition edge failures and cross-partition Fedge-node failover (Section 5.1).
- A deterministic Rule-based Fedge that uses precomputation during the warning window to absorb the decision-latency cost paid by reactive designs, committing cached reassignment plans decisively at failure detection. This achieves zero-stranded recovery and constant-time commit at single-Fedge scale through a single constraint validator that gates every committed action against the per-edge capacity constraint. At cluster scale the same decisive-commit philosophy, applied across cooperating Fedges, is what gives the design its speed and stranding guarantees, and what produces the brief above-nominal load on a receiving Fedge under concurrent cascade commits as its inherent trade-off (Section 5.2).
- An Agentic Fedge in which an LLM deliberative path sits behind the same deterministic triage and constraint validator, producing a structured action and a per-decision natural-language rationale for every committed action (Section 5.3).
- An empirical comparison of the two proposed designs against six reference orchestration policies on the CIFAR-10 image classification task, covering both failure classes through two failure scenarios, and a new reporting metric (the Tier-1 fallback rate) that isolates the reliability of the LLM Coordinator from the correctness of the system as a whole (Section 5.4).
- An ablation of the Tier-3 episodic memory store that isolates the contribution of the memory tier to the Agentic Fedge’s operational metrics, with the training-relevant metrics statistically indistinguishable across the memory-on and memory-off arms (Section 5.4.4).

The remainder of the chapter is organised as follows. Section 5.1 formalises the system model, the training objective and aggregator interface, the failure model and resilience criterion, and the orchestration task as a control problem. Section 5.2 develops the Rule-based Fedge. Section 5.3 develops the Agentic Fedge. Section 5.4 reports the experimental evaluation, and Section 5.5 summarises the chapter.

5.1 Problem Formulation

This section formalises the system over which orchestration acts, the training objective that must be preserved under failure, the failure events together with the resilience criterion that defines acceptable recovery, and the orchestration task itself as a control problem. The formulation is the basis for both designs that follow and is used unchanged in the experimental evaluation later in the chapter.

5.1.1 System Model

A three-tier HFL system is considered, comprising a client tier, an edge tier, and a cloud tier. The client and edge tiers follow the canonical HFL formulation introduced in Chapter 1. The cloud tier is implemented as a set of cooperating nodes rather than a single logical controller; this structural change underpins the remainder of the chapter.

Let $\mathcal{K} = \{1, \dots, K\}$ denote the set of K client devices at the client tier. Each client k holds a local dataset $\mathcal{D}_k$ drawn from a client-specific distribution. Let $\mathcal{E} = \{1, \dots, M\}$ denote the set of M edge nodes at the edge tier, with $M \ll K$. Each edge $e \in \mathcal{E}$ serves a disjoint subset of clients $\mathcal{K}_e \subset \mathcal{K}$ with $\bigcup_{e \in \mathcal{E}} \mathcal{K}_e = \mathcal{K}$, and has a finite capacity κ_e that bounds how many clients it can host. The assignment of clients to edges may change between rounds in response to orchestration actions, so $\mathcal{K}_e(t)$ is used to denote the live client membership of edge e at the start of round t , with $\mathcal{K}_e = \mathcal{K}_e(0)$ at initialisation.

The cloud tier consists of a set of Fedge nodes $\mathcal{F} = \{1, \dots, |\mathcal{F}|\}$, building on the client-edge-Fedge architecture from prior work [45], together with a thin shared state store $\mathcal{S}$. Each Fedge $f \in \mathcal{F}$ governs a disjoint partition of edges $\mathcal{E}_f$, with $\bigcup_{f \in \mathcal{F}} \mathcal{E}_f = \mathcal{E}$, and has a finite edge capacity κ_f^{edge} that bounds how many edges it can host. As with $\mathcal{K}_e(t)$, $\mathcal{E}_f(t)$ denotes the live edge membership of Fedge f at the start of round t . The state store $\mathcal{S}$ persists global model checkpoints, topology metadata, and committed reassignment decisions, but performs no aggregation itself. The hierarchy remains three-tier; only the implementation of the cloud tier changes. The resulting topology satisfies $|\mathcal{F}| \ll M \ll K$, and each client is associated with exactly one edge and one Fedge through the mapping $k \rightarrow e(k) \rightarrow f(e(k))$. Distributing the orchestration role across cooperating Fedges removes the single logical controller from the system: no individual node can halt training by crashing, and decision latency is bounded by partition-local round-trips rather than by the round-trip to a central controller.

5.1.2 Training Objective and Aggregator Interface

Given model parameters $w \in \mathbb{R}^d$ and local empirical risk $F_k(w)$ on client k , the hierarchical objective is

$$\min_w F(w) = \sum_{f \in \mathcal{F}} \sum_{e \in \mathcal{E}_f} \sum_{k \in \mathcal{K}_e} \frac{|\mathcal{D}_k|}{|\mathcal{D}|} F_k(w), \quad (5.1)$$

where $|\mathcal{D}| = \sum_k |\mathcal{D}_k|$. Equation (5.1) extends the two-tier hierarchical objective of Chapter 1 by partitioning the edge tier across Fedges; it reduces to the two-tier form when $|\mathcal{F}| = 1$. Aggregation proceeds in stages. Each edge e computes $w_e = \mathcal{A}(\{w_k\}_{k \in \mathcal{K}_e})$ over the updates from its assigned clients, and each Fedge f computes $w_f = \mathcal{A}(\{w_e\}_{e \in \mathcal{E}_f})$ over the updates from its assigned edges. The aggregation operator $\mathcal{A}$ is generic. In the experiments later in the chapter, $\mathcal{A}$ is instantiated as HierFAVG, but no part of the formulation in this section depends on that choice. The orchestration layer consumes only the standard training signals that $\mathcal{A}$ already produces, and presents only the live membership to the next invocation of $\mathcal{A}$; the choice of aggregator is a parameter of the design rather than a coupling.

5.1.3 Failure Model and Resilience Criterion

The following failure events are considered during training round t . A lowercase ϕ denotes a single-node event, and an uppercase Φ denotes a set of concurrently-failed nodes within the same partition.

Edge failure $\phi_e(t)$. An edge e becomes unresponsive, and its client subset $\mathcal{K}_e$ is stranded. Until the orchestration layer commits a corrective action, the stranded clients contribute nothing to the round.

Cascading edge failure $\Phi_f(t) \subseteq \mathcal{E}_f$. Multiple edges within the same Fedge partition fail over a small number of consecutive rounds. The orphaned client subsets must be redistributed across surviving edges in the same partition, subject to the per-edge capacity κ_e .

Fedge failure $\phi_f(t)$. A Fedge f crashes, and its governed subtree $\mathcal{E}_f$ is orphaned. Recovery requires cross-partition migration: the orphaned edges must be hosted by surviving Fedges, subject to the per-Fedge edge capacity κ_f^{edge} .

Two scenarios in the experimental evaluation exercise these events. Scenario A exercises $\phi_e(t)$ and $\Phi_f(t)$ within a single Fedge partition: edges fail and clients are reassigned across surviving edges in the same partition. Scenario B exercises $\phi_f(t)$ across a multi-Fedge cluster: an entire Fedge sub-cluster crashes and the unit of migration becomes the edge rather than the client. The Fedge-failover case is harder than the edge case in three respects. The unit of placement is coarser, the capacity headroom is tighter because a surviving Fedge typically has capacity for one or two extra edges rather than for an arbitrary mix of clients, and the per-event blast radius is larger. Client-side adversarial behaviour is out of scope for this chapter.

Let w^* denote the global optimum of Equation (5.1) in the absence of failures, and let T_ϵ denote the rounds required to reach $F(w) - F(w^*) \leq \epsilon$. An orchestration scheme is said to be resilient if, given a failure sequence, $T_\epsilon^{\text{fail}} \leq (1 + \delta)T_\epsilon^{\text{clean}}$ for some bounded δ . Two operational quantities are defined to track this property. Let $\mathcal{Z}(t)$ denote the set of clients (Scenario A) or edges (Scenario B) that are not assigned to any live host in round t . The stranded count

$$\Sigma(T) = \sum_{t=1}^T |\mathcal{Z}(t)| \quad (5.2)$$

measures lost training opportunity accumulated over the training horizon. The decision latency Λ measures the number of rounds between detection of a failure and the commit of a corrective action. A resilient scheme keeps Σ bounded and minimises Λ . The resilience inequality $T_\epsilon^{\text{fail}} \leq (1 + \delta)T_\epsilon^{\text{clean}}$ above is the formal version of the failure-handling objective set out in Section 1.4 of Chapter 1; the rest of this chapter develops two orchestration designs that satisfy it and an empirical evaluation that tests how tightly each design binds it.

5.1.4 Orchestration as a Control Problem

Each Fedge runs a local control loop over its governed partition, following the observe-classify-decide-act pattern established for cooperating controllers in cloud-native systems [58]. The loop is defined by four objects: a per-round observation, a per-node health classification, a bounded action space, and a deterministic safety floor that gates every committed action.

Local observation. At each round t , Fedge f collects

$$o_f(t) = (h_f(t), \ell_f(t), \tau_f(t)), \quad (5.3)$$

comprising heartbeats $h_f(t)$, loss trajectories $\ell_f(t)$, and latency measurements $\tau_f(t)$ over the nodes in its partition. The observation is local: no observation from $\mathcal{E}_{f'}$ for $f' \neq f$ is required for within-partition events. This locality is what makes the distributed design possible; each Fedge can decide for its own partition without consulting the others.

Per-node health classification. The observation is mapped to a per-node health label drawn from a four-state classification, in the spirit of unreliable failure detectors in distributed systems [59, 60]:

$$\mathcal{Q} = \{\text{HEALTHY}, \text{DEGRADING}, \text{AMBIGUOUS}, \text{FAILED}\}. \quad (5.4)$$

A missing heartbeat beyond a configurable window maps to **FAILED**. Sustained loss or latency deviation without a missed heartbeat maps to **DEGRADING**. Signals that match neither pattern cleanly are flagged **AMBIGUOUS** and trigger deliberation rather than being held for the next round. Four states are distinguished from three independent signals, rather than collapsed onto a single alarm. The corrective action is therefore selected at the granularity of the actual failure mode: immediate reassignment for **FAILED**, anticipatory pre-warming for **DEGRADING**, and escalation for **AMBIGUOUS**, rather than fired indiscriminately on a single threshold crossing.

Action space. The control loop selects from

$$\mathcal{U} = \{\text{NOOP}, \text{PRE_WARM}, \text{REASSIGN}\}. \quad (5.5)$$

A **REASSIGN** action commits a list of migrations $\{(\text{child}, \text{source}, \text{destination})\}$, where the child is a client in Scenario A and an edge in Scenario B, subject to capacity constraints at the destination. A **PRE_WARM** action reserves capacity on listed candidate hosts in anticipation of further degradation. A **NOOP** action leaves the live membership unchanged.

Safety floor. Every committed action passes through a constraint validator V . The validator enforces four invariants. Capacity: $|\mathcal{K}_e(t+1)| \leq \kappa_e$ in Scenario A, and $|\mathcal{E}_f(t+1)| \leq \kappa_f^{\text{edge}}$ in Scenario B. Liveness: the destination node is alive at the time of commit. Uniqueness: each child appears at most once across the migration list. Self-preservation: source and destination are distinct for every migration. If a proposed action violates V , a corrective retry is issued with the violation list as feedback. On a second consecutive failure, a deterministic fallback policy commits a capacity-aware nearest-with-room placement. The four invariants therefore hold for every committed action, regardless of how the action was produced. The validator is the single point at which correctness is checked.

The control loop must satisfy three properties. Distribution: each Fedge decides over its own partition. Locality: in-scope decisions for partition $\mathcal{E}_f$ use only $o_f(t)$ and locally cached state. Separation: the inner aggregator $\mathcal{A}$ is invoked unchanged over whatever live membership the control loop presents to it. The two designs developed in the following sections both satisfy these properties and differ only in how the per-node health classification is computed and how an action is selected when the classification is non-trivial.

5.2 Rule-based Fedge

This section presents the first of the two proactive orchestration designs developed in this chapter. The Rule-based Fedge commits a precomputed reassignment plan deterministically, with the constraint validator V of Section 5.1.4 checking every committed action. The second design, the Agentic Fedge, is presented in Section 5.3.

5.2.1 Design Principle

A baseline orchestration design for HFL must be predictable, fast in real time, and free of external dependencies. A deterministic rule-based control loop meets all three requirements. Its decision logic consists of a finite set of branches over a small state space, so the per-round commit completes in constant time, and the behaviour of the system can be reproduced from the failure schedule and the configuration alone. No model inference, no external service call, and no learned component is required during the critical path. The decision-latency cost that is usually paid by a reactive system is eliminated in this design through precomputation during the DEGRADING window. When a node enters DEGRADING, a capacity-aware nearest-with-room reassignment plan is computed in the background and stored under the node identifier. When the same node transitions to FAILED, the cached plan is committed without further work. The corrective action is therefore prepared ahead of the failure rather than after it, and is ready at the moment it is needed.

5.2.2 Architecture

The Rule-based Fedge consists of four cooperating modules, connected as shown in Figure 5.1. The planning work is performed by the Knowledge Module during DEGRADING windows so that the per-round action selection by the Overlay Controller remains constant-time.

Failure Detector. The Failure Detector uses the local observation defined in Equation (5.3) and produces a per-node health label drawn from $\mathcal{Q}$. The classification uses three independent signals and one fixed threshold per signal. The heartbeat signal $h_f(t)$ provides the strongest evidence: a node whose number of missed heartbeats exceeds a configured window is labelled FAILED, regardless of the other two signals. The latency signal $\tau_f(t)$ provides a second basis for the same label, since a node that cannot complete its share of round work within the round budget is effectively unavailable. The loss signal $\ell_f(t)$ provides the remaining two labels: a sustained deviation above the DEGRADING threshold yields the DEGRADING label, and a larger deviation above the AMBIGUOUS threshold yields the AMBIGUOUS label. A node whose signals all remain below their respective DEGRADING thresholds is labelled HEALTHY. The threshold values are deferred to the experimental setup later in this chapter so that the design discussion is independent of the experimental configuration. Classification cost is constant per node per round, and the rolling baselines are maintained as exponentially weighted moving averages, in the manner used by accrual-style failure detectors [60].

Knowledge Module. The Knowledge Module has two responsibilities. The first is to maintain the cluster topology view: the live edge set, the live Fedge set, the per-node capacity headroom, and the live memberships $\mathcal{K}_e(t)$ and $\mathcal{E}_f(t)$. The topology view is updated incrementally on every committed action and is the authoritative reference for the cluster state visible to this Fedge. The second responsibility is to maintain the cache of precomputed reassignment plans. When a node n enters DEGRADING, a placement plan is computed for the children that would be orphaned if n transitions to FAILED. The plan is a list of (child, n , destination) tuples. The destination for each child is chosen by a capacity-aware nearest-with-room policy of the kind used by cloud-native schedulers [58]: among the surviving hosts in the partition that have residual capacity, the host with the closest match to the orphan's existing context is selected. Context is class-distribution affinity for Scenario A, where the children are clients, and topological proximity for Scenario B, where the children

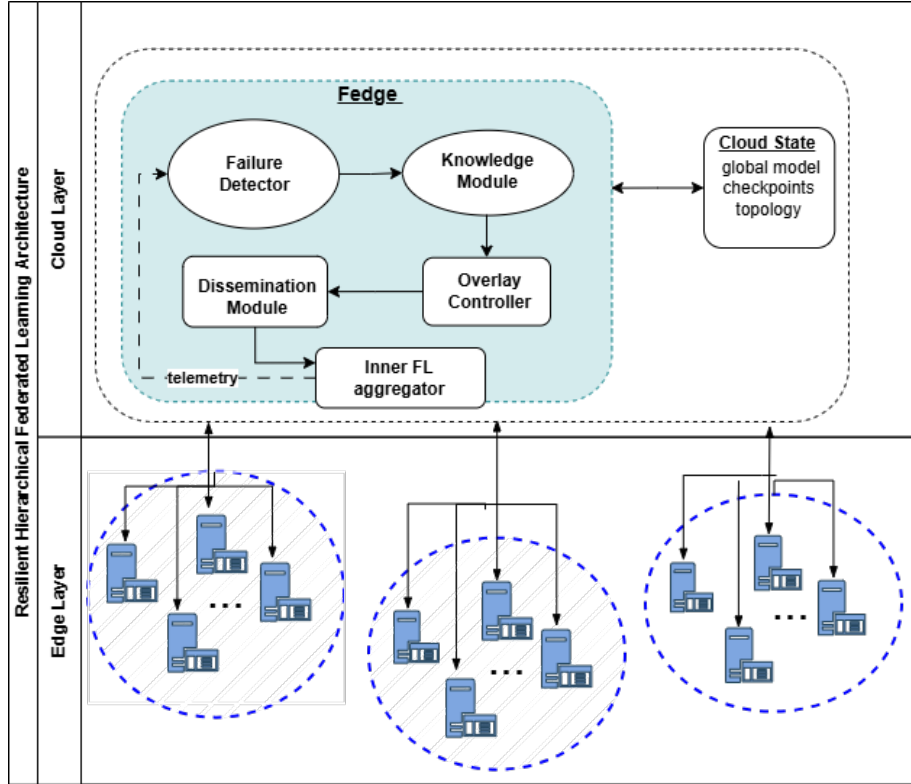


FIGURE 5.1: Rule-based Fedge architecture. Each Fedge in the cloud layer runs four cooperating modules: a Failure Detector, a Knowledge Module, an Overlay Controller, and a Dissemination Module. Telemetry from the edges is consumed by the Failure Detector, and the live membership is presented to the inner aggregator by the Dissemination Module. Cluster topology and global model checkpoints are persisted to the shared state store $\mathcal{S}$.

are edges. Ties are broken by remaining capacity. The plan is indexed by node n and held in a small in-memory cache local to the Fedge. If n later returns to HEALTHY without transitioning to FAILED, the cached plan is discarded. If n transitions to FAILED, the cached plan is passed to the Overlay Controller for commit. Because the plan is computed during the DEGRADING window, surviving capacity is still nominal and the topology has not yet been disrupted. The placement decisions are therefore informed by the steady-state context rather than by the disturbed post-failure state.

Overlay Controller. The Overlay Controller is the decision-making module of the design: it chooses the action to take on each round and passes it to the Dissemination Module for execution. On each round it consults the Failure Detector for transitions in the per-node label and selects one of three branches. The precomputed fast path is taken when a node has transitioned to FAILED and the Knowledge Module holds a cached plan for that node. The cached plan is committed: it is sent once to the Dissemination Module, and the topology view is updated once. No fresh planning is performed. The reactive fallback is taken when a node has transitioned to FAILED but no cached plan exists. This case arises when the failure was instantaneous, so the DEGRADING window was skipped. A fresh capacity-aware nearest-with-room placement is computed at commit time, using the current topology view. The branch incurs a one-round latency penalty: the planning step is performed within the round of detection rather than ahead of it. The third branch is

the trivial-case fast path. When no transition has occurred and the partition state is uneventful, the action committed is NOOP. When one or more nodes carry the DEGRADING label but none has yet transitioned to FAILED, the action committed is also NOOP at the level of membership, and the Knowledge Module continues pre-computation in the background. Every committed action is gated by the constraint validator V of Section 5.1.4 before being passed to the Dissemination Module. For the Rule-based Fedge, the validator catches a small class of rare stale-cache cases. The most common case is a precomputed plan whose destination has lost its capacity headroom between the time the plan was cached and the time of commit, because another Fedge committed an action that filled the destination's spare capacity in the meantime. When this occurs, the plan is rerun on the current topology view and the corrected action is committed in the same round.

Dissemination Module. The Dissemination Module propagates the committed action to the inner aggregator $\mathcal{A}$ and updates the membership for the next round. For each migration tuple (child, n , destination) in the committed plan, the child's association is updated from n to the destination in the local topology view, and the update is forwarded to the shared state store $\mathcal{S}$ for cross-Fedge visibility. The inner aggregator $\mathcal{A}$ is then called over the updated live membership for the next training round. The Dissemination Module performs no planning of its own.

5.2.3 Per-Round Control Loop

Algorithm 3 shows the steps taken in each round. The four modules run in sequence. The Failure Detector assigns a health label to each child node in the partition: an edge node in the within-partition case, or a peer Fedge node in the cross-partition case. The Knowledge Module then updates its plan cache: a new placement plan is computed for any node that has just entered the DEGRADING state, and any cached plan for a node that has returned to HEALTHY is removed. The Overlay Controller selects the action for the round. If no node has just transitioned to FAILED, a NOOP is committed. If one or more nodes have just transitioned to FAILED, their migrations are collected into a single combined action: each just-failed node contributes the cached plan when one exists, or a freshly computed plan otherwise. The Dissemination Module then sends the committed action to the inner aggregator and updates the live membership for the next training round.

5.3 Agentic Fedge

This section presents the second of the two proactive orchestration designs developed in this chapter. The Agentic Fedge commits an action produced through deliberation, with the same constraint validator V of Section 5.1.4 checking every committed action, and logs a short natural-language rationale alongside each one.

5.3.1 Design Principle

The Agentic Fedge adopts the same control-loop structure of Section 5.1.4: the local observation $o_f(t)$, the four-state classification $\mathcal{Q}$, the bounded action space $\mathcal{U}$, and the constraint validator V . It differs from the Rule-based Fedge in how the per-node classification is turned into an action. Whereas the Rule-based Fedge commits a precomputed plan deterministically, the Agentic Fedge produces an action by deliberation using an LLM, and logs a short natural-language rationale alongside every

Algorithm 3 Rule-based Fedge: per-round control loop on Fedge f .**Require:** Local observation $o_f(t)$; topology view $\mathcal{T}_f$; plan cache $\mathcal{P}_f$; constraint validator V

```

1:  $L \leftarrow \text{FAILUREDETECTOR}(o_f(t))$  ▷ label each node in the partition
2: for all  $n$  in partition do
3:   if  $L[n] = \text{DEGRADING}$  and  $n \notin \mathcal{P}_f$  then
4:      $\mathcal{P}_f[n] \leftarrow \text{CAPACITYAWAREPLAN}(n, \mathcal{T}_f)$ 
5:   else if  $L[n] = \text{HEALTHY}$  and  $n \in \mathcal{P}_f$  then
6:     remove stale entry  $\mathcal{P}_f[n]$ 
7:   end if
8: end for
9:  $\mathcal{N}_{\text{fail}} \leftarrow \{n : L[n] = \text{FAILED} \text{ and previous label} \neq \text{FAILED}\}$  ▷ nodes that just transitioned
10: if  $\mathcal{N}_{\text{fail}} = \emptyset$  then
11:    $a \leftarrow \text{NOOP}$ 
12: else
13:    $a.\text{migrations} \leftarrow \emptyset$ 
14:   for all  $n \in \mathcal{N}_{\text{fail}}$  do
15:     if  $\mathcal{P}_f[n]$  exists then
16:        $a.\text{migrations} \leftarrow a.\text{migrations} \cup \mathcal{P}_f[n].\text{migrations}$ 
17:     else
18:        $a.\text{migrations} \leftarrow a.\text{migrations} \cup \text{CAPACITYAWAREPLAN}(n, \mathcal{T}_f).\text{migrations}$ 
19:     end if
20:   end for
21:   if  $V(a) \neq \emptyset$  then
22:      $a \leftarrow \text{CAPACITYAWAREPLAN}(\mathcal{N}_{\text{fail}}, \mathcal{T}_f)$  ▷ stale-cache retry on current topology with combined orphans
23:   end if
24: end if
25:  $\text{DISSEMINATE}(a, \mathcal{T}_f, \mathcal{S})$  ▷ update membership and forward to  $\mathcal{A}$ 
26: return  $a$ 

```

committed action. Two operational settings motivate this choice. The first is a deployment in which the failure modes are not fully known in advance, or in which the rule table is updated infrequently relative to the rate at which new patterns appear; the deliberative path can reason about patterns the rule table does not cover. The second is a deployment in which operators require a per-decision rationale, for example to justify a reassignment to an internal review process or to attribute an unexpected accuracy regression to a specific orchestration action; the deliberative path supports both use cases through its logged rationale, while the Rule-based Fedge supports neither. The validator V gates the LLM's output in the same way that it gates the rule-based output, so the four invariants of capacity, liveness, uniqueness, and self-preservation are preserved regardless of what the LLM emits.

5.3.2 Architecture

The architecture is organised as three tiers, shown in Figure 5.2. The base of the architecture is a deterministic triage that handles routine rounds without invoking the LLM, paired with the same constraint validator V . Above the triage is the LLM

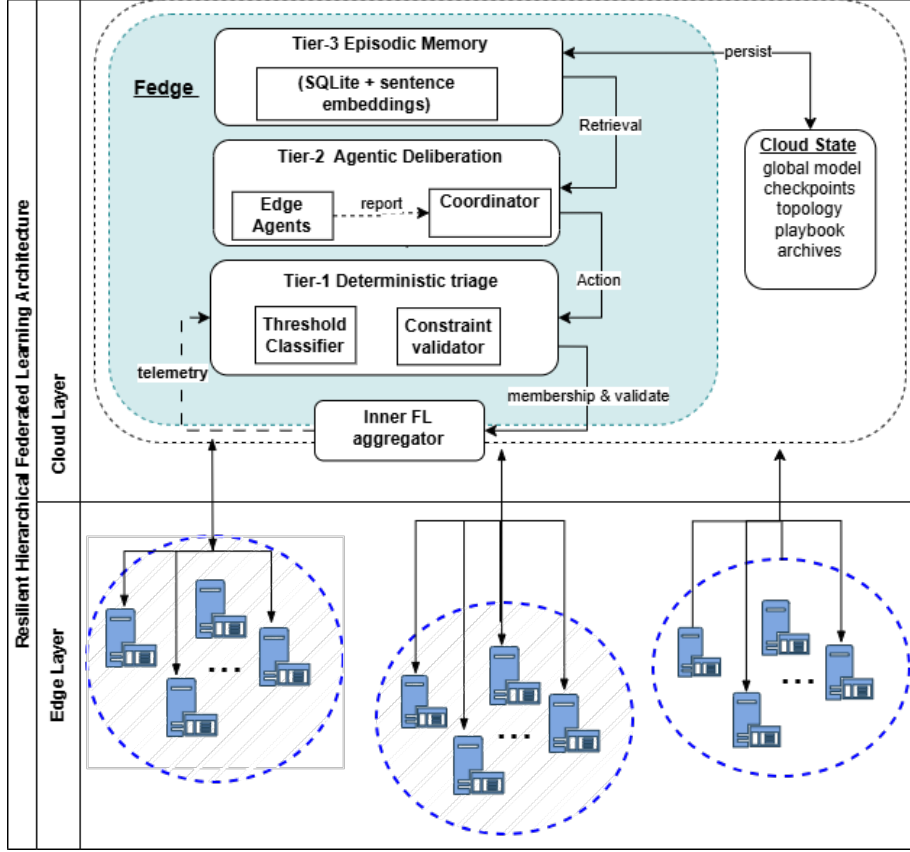


FIGURE 5.2: Agentic Fedge architecture. Tier 1 performs deterministic triage and constraint validation; Tier 2 is the LLM deliberative path, called in three role-conditioned passes (Edge Agents and Coordinator) to produce a structured action; Tier 3 is an episodic memory store queried by the Coordinator during deliberation. Edge telemetry flows into Tier 1, and the validated action is dispatched to the inner aggregator. Cluster state is persisted to the shared state store.

deliberative path, in which a single model instance is called in three role-conditioned passes to produce a structured action. At the top is an episodic memory store that accumulates compact summaries of past episodes and retrieves them by similarity during deliberation. The three tiers are connected so that the triage dispatches the routine rounds, escalates the non-trivial rounds to the deliberative path, and queries the memory store during deliberation; the validator gates every committed action regardless of which tier produced it.

Tier 1: Deterministic Triage and Constraint Validator. The Tier-1 path uses only thresholds and integer arithmetic and never invokes the LLM. It carries three responsibilities. The first is the per-node health classification described in Section 5.1.4, run on every round in constant time. The second is a trivial-case fast path that handles routine rounds without escalation. When the partition contains no AMBIGUOUS nodes, no contested capacity, and at most one FAILED node, Tier 1 commits a deterministic action directly: a NOOP when no FAILED node is present, or a capacity-aware nearest-with-room placement when exactly one FAILED node is present and sufficient cluster capacity is available. The third responsibility is the constraint validator V , which gates every committed action regardless of whether it originates in Tier 1 or Tier 2. Routine rounds therefore bypass the LLM entirely through the

trivial-case fast path: no model tokens are consumed, a constant number of comparisons per node is required, and the Agentic Fedge’s correctness floor is established at this tier.

Tier 2: LLM Deliberative Path. Tier 2 is invoked when Tier 1 escalates: when an AMBIGUOUS node is present, when multiple failures arrive within a short window, or when cluster capacity is contested. The deliberative path is implemented as a sequence of three role-conditioned calls on a single model instance, rather than as message passing between separate processes. The three roles share weights and are distinguished only by the system prompt and the input payload. The pattern adapts the coordinator-worker decomposition that has emerged from the recent LLM-agent literature [61–63]. The worker-coordinator messaging is compressed into role-conditioned calls on a single model, for cost and latency reasons. Every output is validated against a Pydantic schema with a single corrective retry on parse failure [64, 65].

The *Steward* call is the first role-conditioned pass. A single batched LLM call produces $|\mathcal{E}_f|$ structured reports, one per node in the partition. Each report contains a health label, a confidence score in $[0, 1]$, and a short rationale citing a specific signal from the local observation. Inference is batched into one generation for cost efficiency. The system prompt is a stable prefix; the per-round payload is a compact JSON of node telemetry.

The *Bidding* call is the second role-conditioned pass and is issued only when at least one FAILED node has orphaned children. The call collects per-host bids: each healthy host returns how many children it can accept (bounded by its residual capacity), an affinity score reflecting class-distribution overlap with the orphans, and a short rationale. The pattern is a Contract-Net call for proposals in the sense of the multi-agent systems literature [66], but the bids are emitted by the same model instance running the Steward role on a different prompt rather than by independent processes.

The *Coordinator* call is the third and final pass. It receives the per-node reports, the bids when present, and the top- k playbooks retrieved from Tier 3, together with a compact capacity map. It emits a single structured action containing the action type, the list of migrations, and a two-to-three-sentence rationale citing specific reports and bids. The rationale is logged to disk for audit and is the per-decision explanation that the rule-based design does not produce.

The validation loop completes the Tier-2 sequence. The Coordinator’s action is checked by V . On a violation, a corrective retry sends the violation list back to the Coordinator and requests a corrected action. On a second consecutive violation, a deterministic fallback policy commits a capacity-aware nearest-with-room placement, and the rationale is logged as a fallback event rather than as an LLM-emitted action. The system is therefore correct with respect to the four invariants regardless of whether the LLM emits a constraint-valid action.

Tier 3: Episodic Memory. Tier 3 accumulates compact summaries of prior episodes. At the end of each episode, a summarisation prompt produces a compact playbook tuple of the form (situation, action, outcome, score), where the score in $[0, 1]$ rates the outcome quality. Each playbook is embedded with a sentence-transformer [67] and stored in a SQLite table.

During Tier-2 deliberation, the Coordinator queries the top- k playbooks ranked by a weighted combination of cosine similarity and outcome score. The weight is set so that cosine similarity dominates while the outcome score acts as a tiebreaker:

$$\text{rank}(p) = 0.7 \cos(q, p) + 0.3 \text{score}(p), \quad (5.6)$$

where q is the embedding of the current situation summary and p ranges over stored playbooks. The retrieved playbooks become in-context examples in the Coordinator prompt, allowing past episodes to shape current decisions without gradient updates to the model.

The memory tier also supports a memory pre-check that runs before Tier-1 triage. If a stored playbook with high situation similarity (≥ 0.85) and low outcome score (≤ 0.5) is found, a memory flag forces escalation to Tier 2 even when Tier 1 would otherwise dispatch a fast-path action. Through this pre-check, previously-observed bad outcomes shape future decisions: a fast-path action that produced a poor outcome on a similar past situation is not committed without review by the LLM.

5.3.3 Per-Round Control Loop

Algorithm 4 shows the steps taken in each round. The Tier-1 triage assigns a health label to each child node in the partition (an edge node in the within-partition case, or a peer Fedge node in the cross-partition case) and inspects the partition state. On routine rounds the Tier-1 fast path commits a deterministic action directly, in the same way as the Rule-based Fedge. On all other rounds the loop escalates to Tier 2. The memory store is queried for relevant playbooks, the three role-conditioned LLM calls are then issued in sequence (Steward, optional Bidding, Coordinator), and the Coordinator’s action is checked by V . A constraint violation triggers a corrective retry. A second consecutive violation passes control to the deterministic fallback. Every committed action, regardless of which path produced it, is checked by V and sent to the inner aggregator for the next training round.

5.4 Experimental Evaluation

This section evaluates the two designs developed in this chapter against six reference orchestration policies and isolates the contribution of the Tier-3 episodic memory store through an ablation. The evaluation is organised around three subsections that exercise the two failure classes of Section 5.1.3. Section 5.4.2 reports within-partition recovery (Scenario A), in which edges fail and clients are reassigned across surviving edges inside a single Fedge. Section 5.4.3 reports cross-partition failover (Scenario B), in which an entire Fedge crashes and its edges migrate to surviving Fedges. Section 5.4.4 reruns the Scenario A high-intensity configuration with the Tier-3 memory store disabled, holding every other component fixed, and attributes the Agentic Fedge’s operational metrics to the memory tier rather than to the architecture as a whole. All three subsections share the setup described in Section 5.4.1; the threshold values deferred from Section 5.2 and Section 5.1.4 are fixed there.

5.4.1 Experimental Setup

The experimental evaluation tests the resilience criterion of Section 5.1.3, that an orchestration scheme keeps the round overhead $T_e^{\text{fail}} \leq (1 + \delta)T_e^{\text{clean}}$ bounded under failure, by exercising the two failure classes of Section 5.1.3 as two scenarios. Scenario A exercises within-partition recovery: edges fail and clients are reassigned across surviving edges inside a single Fedge. Scenario B exercises cross-partition failover: an entire Fedge crashes and its edges migrate to surviving Fedges. The two scenarios share the workload and the failure-model instantiation; they differ in the system configuration and the failure horizon. The setup is described once below and held fixed across all methods.

Algorithm 4 Agentic Fedge: per-round control loop on Fedge f .

Require: Local observation $o_f(t)$; topology view $\mathcal{T}_f$; capacity map C ; memory store $\mathcal{M}$; constraint validator V

```

1:  $L \leftarrow \text{FAILUREDETECTOR}(o_f(t))$   $\triangleright$  Tier-1: label each node in the partition
2: if  $\text{TRIVIAL}(L, C)$  and not  $\text{MEMORYESCALATE}(\mathcal{M}, L)$  then
3:    $a \leftarrow \text{DETERMINISTICDISPATCH}(L, \mathcal{T}_f)$   $\triangleright$  Tier-1 fast path
4: else
5:    $\text{reports} \leftarrow \text{LLMREPORTS}(\{s_n\})$   $\triangleright$  Steward call: batched per-node reports
6:    $\text{playbooks} \leftarrow \mathcal{M}.\text{RETRIEVE}(\text{SUMMARY}(\text{reports}), k = 3)$ 
7:   if any node in  $L$  is FAILED with orphans then
8:      $\text{bids} \leftarrow \text{LLMBIDS}(\text{HEALTHYHOSTS}, \text{ORPHANS}, \text{AFFINITY})$   $\triangleright$  Bidding call
9:   else
10:     $\text{bids} \leftarrow \emptyset$ 
11:   end if
12:    $a \leftarrow \text{LLMCOORDINATOR}(\text{reports}, \text{bids}, \text{playbooks})$   $\triangleright$  Coordinator call
13:   if  $V(a) \neq \emptyset$  then
14:      $a \leftarrow \text{LLMCOORDINATOR}(\dots, \text{violations} = V(a))$   $\triangleright$  corrective retry
15:     if  $V(a) \neq \emptyset$  then
16:        $a \leftarrow \text{DETERMINISTICFALLBACK}(\text{ORPHANS}, C)$   $\triangleright$  capacity-aware nearest-with-room
17:     end if
18:   end if
19: end if
20:  $\text{DISSEMINATE}(a, \mathcal{T}_f, \mathcal{S})$   $\triangleright$  update membership and forward to  $\mathcal{A}$ 
21: return  $a$ 

```

Workload and inner aggregator. The image classification task is CIFAR-10 (50,000 training and 10,000 test images). The inner model is a LeNet-style convolutional network ($\approx 62k$ parameters). Training proceeds for $T = 200$ global rounds. In each round, every participating client performs five local epochs of mini-batch stochastic gradient descent with batch size 64, initial learning rate 10^{-2} with geometric decay $\gamma = 0.995$, momentum 0.9, weight decay 5×10^{-4} , and gradient-norm clipping at 1.0. The inner aggregator $\mathcal{A}$ is instantiated as HierFAVG [13] at every level of the hierarchy. The orchestration layer consumes only the training signals $\mathcal{A}$ already produces and presents only the live membership back to $\mathcal{A}$, in the sense established in Section 5.1.2.

Scenario A: within-partition system configuration. Scenario A uses a single Fedge governing $M = 10$ edges and $K = 100$ clients, with ten clients per edge at initialisation. The per-edge capacity is $\kappa_e = 25$ at high failure intensity and $\kappa_e = 20$ at medium failure intensity. The data are partitioned with a two-level hierarchical Dirichlet scheme ($\alpha_{\text{server}} = \alpha_{\text{client}} = 0.5$), which yields a moderate non-IID distribution at both the edge and client tiers. Scenario A is run at two failure intensities, *medium* (MTBF = 570) and *high* (MTBF = 250), both with a failure horizon of 150 rounds.

Scenario B: cross-partition system configuration. Scenario B uses a multi-Fedge cluster of ten Fedges, each governing four edges with four clients per edge ($|\mathcal{F}| = 10$, $M = 40$, $K = 160$), with a per-Fedge edge-capacity ceiling $\kappa_f^{\text{edge}} = 8$. The CIFAR-10 partition is extended to a three-level hierarchical Dirichlet scheme ($\alpha_{\text{cluster}} =$

$\alpha_{\text{server}} = \alpha_{\text{client}} = 0.5$). The failure model samples a fixed number of Fedge failures from a per-Fedge exponential distribution with mean MTBF = 350, over a failure horizon of 175 rounds. This configuration is a strict superset in difficulty: the unit of migration is a whole edge rather than a client, the surviving capacity headroom is tight (a surviving Fedge typically has room for one or two extra edges, not an arbitrary mix of clients), and each Fedge failure has a larger blast radius.

Failure-model instantiation. In both scenarios, failures are sampled from independent per-node Poisson processes. For each node, the time to failure is drawn from an exponential distribution with mean MTBF and clipped to the scenario’s failure horizon. Each failure occurs as an instantaneous crash with probability $p_{\text{hard}} = 0.3$, or otherwise as a degradation ramp of 10 rounds over which the per-node tick cost rises from baseline to three times baseline. The failure trajectory is held fixed across methods within a run, so any difference between methods is attributable to the orchestration policy and not to a different realisation of the failures.

Baselines. The two proposed designs are compared against six reference policies that span the design space of recovery from failures, grouped by what each policy tests for. The first two define the bounds: *Clean* runs the system with no failures injected and is the upper bound on training quality; *Drop* abandons orphaned children and is the lower bound that measures what is at stake. The next two are common reassignment heuristics: *Naive* reassigns orphans uniformly at random subject to capacity, and *Nearest* reassigns to the most similar surviving host with capacity checked but not balanced. The fifth, *Centralised*, is a cloud-coordinator policy with a three-round detection delay and capacity-aware random scatter, analogous to default Kubernetes scheduling. The sixth is the *Rule-based Fedge* of Section 5.2. The seventh method is the *Agentic Fedge* of Section 5.3. All seven methods share identical training, partitioning, and failure-injection configurations.

Resilience metrics. Two operational metrics from Section 5.1.3 are reported as the primary resilience measurements. The decision latency Λ (rounds) is the mean number of rounds between failure detection and the commit of a corrective action, and is the quantity a resilient scheme is required to minimise. The stranded count Σ is the accumulated client-rounds (Scenario A) or edge-rounds (Scenario B) of unassigned children, as defined in Equation (5.2), and is the quantity a resilient scheme is required to keep bounded. Three further operational metrics audit the capacity bounds and the commit behaviour: *migration* (the mean number of children moved per REASSIGN event) audits churn; *maximum post-failure load* (the largest population on any surviving host after reassignment) audits the capacity ceiling; and the *real-time decision rate* is the fraction of REASSIGN events committed in the same round as detection ($\Lambda = 0$). Three additional metrics are specific to the Agentic Fedge: the *Tier-1 fallback rate* (the fraction of agentic rounds whose committed action came from the deterministic fallback after a second LLM constraint-validation failure), the *total constraint violations* rejected by V , and the *per-decision natural-language rationale* logged for audit.

Agentic configuration. The Tier-2 deliberative path uses Qwen2.5-7B-Instruct [68], AWQ-quantised to four bits [69], served through vLLM [70] with a maximum context of 4096 tokens, a generation budget of 1024 tokens, and greedy decoding at temperature zero for deterministic output. Structured output is enforced against a Pydantic schema with a single corrective retry on parse failure [64, 65]. The Tier-1 thresholds deferred from Section 5.2 are fixed as follows: a latency multiplier of $1.5\times$ the rolling baseline maps to DEGRADING and $3.0\times$ to FAILED; one missed heartbeat

TABLE 5.1: Scenario A operational metrics at medium failure intensity (MTBF = 570, $\kappa_e = 20$). Mean $\pm$ standard deviation over independent runs. Bold marks the best value per column among the failover methods.

Method	$\Lambda \downarrow$ (rounds)	Migration (clients/event)	Max load $\downarrow$ (clients)	Real-time $\uparrow$ (frac.)	Stranded $\downarrow$ (client-rds)	Audit
Clean	n/a	n/a	10.0 $\pm$ 0.0	n/a	0	no
Drop	n/a	0	10.0 $\pm$ 0.0	n/a	4733 $\pm$ 612	no
Naive	1.00	11.2 $\pm$ 1.3	17.3 $\pm$ 0.9	0.0	0	no
Nearest	1.00	16.0 $\pm$ 10.2	43.3 $\pm$ 4.7	0.0	0	no
Centralised	3.00	11.3 $\pm$ 1.5	17.7 $\pm$ 1.7	0.0	113 $\pm$ 21	no
Rule-based Fedge	0.20	11.0 $\pm$ 3.0	20.0 $\pm$ 0.0	0.80	0	no
Agentic Fedge	0.00	4.83 $\pm$ 1.62	24.6 $\pm$ 0.9	1.00	38 $\pm$ 70	yes

Reroute events: Agentic Fedge 96.8 $\pm$ 38.6, against ~ 3 for the Rule-based Fedge; the elevated count reflects the LLM emitting proactive migrations on DEGRADING and AMBIGUOUS state.

maps to DEGRADING and three to FAILED; a loss deviation of 0.10 above an exponentially weighted baseline ($\alpha = 0.1$) maps to DEGRADING and 0.50 to AMBIGUOUS; and Tier 1 escalates rather than dispatching the fast path when residual cluster capacity falls below 0.20 of the total. The Tier-3 memory store embeds playbooks with a sentence-transformer [67], ranks retrieval by $0.7 \cos + 0.3 \text{ score}$, and raises the memory-escalation flag when a stored playbook has situation similarity ≥ 0.85 and outcome score ≤ 0.5 .

Reproducibility and hardware. Each method in Scenarios A and B is repeated over multiple independent runs and reported as mean $\pm$ standard deviation. The memory ablation uses an asymmetric allocation between the memory-on and memory-off arms, with the smaller arm justified by the effect sizes observed; the allocation is discussed in Section 5.4.4. All runs are executed on a single NVIDIA L4 24 GB GPU, the largest GPU tier accessible for this work, which constrains the deliberative path to a 7B-class quantised model. The failure schedule, prompt templates, partition cache, and configuration files are fixed across all runs so that any difference between methods is attributable to the orchestration policy and not to a different realisation of the failures.

5.4.2 Within-Partition Recovery

This subsection reports recovery from edge-level and cascading-edge failures within a single Fedge partition, under the Scenario A configuration of Section 5.4.1. The headline results are reported in Table 5.1 for medium intensity and Table 5.2 for high intensity.

Decision latency and capacity. The Agentic Fedge attains $\Lambda = 0.00$ at both intensities, committing the corrective action within the round of detection regardless of whether the failure was preceded by a DEGRADING window. The Rule-based Fedge attains $\Lambda = 0.20$ to 0.21 through precomputation: roughly four-fifths of events consume a plan cached during the prior DEGRADING window, and the remainder (predominantly the instantaneous-crash branch) fall back to one-round reactive computation. Both designs are sub-round; the difference is operationally negligible at the round granularity and comes at the cost of a wall-clock asymmetry discussed below.

The capacity result separates Nearest from the two proposed designs. Nearest reaches a worst-edge load of 43.3 at medium and 56.7 at high, well above the design ceiling, because it selects the most similar survivor without inspecting residual

TABLE 5.2: Scenario A operational metrics at high failure intensity (MTBF = 250, $\kappa_e = 25$). Mean $\pm$ standard deviation over independent runs.

Method	$\Lambda \downarrow$ (rounds)	Migration (clients/event)	Max load $\downarrow$ (clients)	Real-time $\uparrow$ (frac.)	Stranded $\downarrow$ (client-rds)	Audit
Clean	n/a	n/a	10.0 ± 0.0	n/a	0	no
Drop	n/a	0	10.0 ± 0.0	n/a	6323 ± 1792	no
Naive	1.00	13.6 ± 3.5	23.3 ± 4.2	0.0	0	no
Nearest	1.00	14.3 ± 9.0	56.7 ± 9.4	0.0	0	no
Centralised	3.00	12.9 ± 3.5	22.3 ± 1.9	0.0	174 ± 63	no
Rule-based Fedge	0.21	12.5 ± 4.9	25.0 ± 0.0	0.79	0	no
Agentic Fedge	0.00	4.96 ± 0.59	25.0 ± 0.0	1.00	1601 ± 2349	yes

Reroute events: Agentic Fedge 46.2 ± 30.0 , against ~ 5 for the Rule-based Fedge.

capacity. The two proposed designs both gate every committed action through the constraint validator V defined in Section 5.1.4. Within the two proposed designs, the Rule-based and Agentic Fedge make different choices for the cap value the validator is configured to enforce, and the difference is a design choice rather than an implementation accident. The Rule-based path is given an intensity-conditioned conservative cap ($\kappa_e = 20$ at medium, $\kappa_e = 25$ at high), reflecting its precomputation philosophy: a cached plan can become stale before commit, so the conservative cap leaves slack to absorb the divergence and avoid a post-failure overshoot. The Agentic path is given a single protocol-level cap ($\kappa_e = 25$, the per-edge system maximum the underlying hardware can serve), reflecting the deliberative philosophy: the LLM Coordinator decides at commit time how aggressively to consolidate orphaned clients within that limit, conditioning on the failure pattern of the actual round. At high intensity the two configurations coincide and both designs bind exactly at 25. At medium intensity the Rule-based Fedge binds its conservative 20-client cap exactly (20.0 ± 0.0), while the Agentic Fedge reaches up to the protocol-level ceiling on a subset of rounds (24.6 ± 0.9 , with the maximum-load round of four of the runs hitting 25). The architectural floor is the same in both cases. The validator gates every committed action in the sense of Section 5.1.4. What differs is the cap the validator was configured to enforce. The medium-intensity max-load gap of roughly 4.6 clients is the visible signature of the deliberative path using the slack the rigid pre-allocation path declines, which is the design distinction this chapter sets out to test.

Stranded count and churn. The Rule-based Fedge accumulates zero stranded client-rounds at both intensities, because its precomputed plan covers the orphaned clients in the round of the transition. The Agentic Fedge accumulates 38 ± 70 at medium and 1601 ± 2349 at high. The high-intensity figure has wide variance: most runs strand nothing, while a minority see partial-coverage migrations on cascade rounds in which the LLM Coordinator emits a constraint-valid but incomplete REASSIGN, with the residual orphans placed over subsequent rounds that count toward Σ . The Agentic Fedge also produces materially higher migration churn, driven by proactive moves on DEGRADING state where the rule-based path commits a NOOP.

Decision audit and Tier-1 fallback. The Agentic Fedge is the only method that records a natural-language rationale for every committed action. At the model size tested, the rationales are typically terse and policy-restating rather than richly argued; the audit *channel* is uniquely available, while the *depth* of recorded reasoning scales with the underlying model. The Tier-1 fallback rate at the edge level is $6.3\% \pm 9.5\%$ at medium intensity and $9.4\% \pm 9.2\%$ at high intensity: roughly nine in

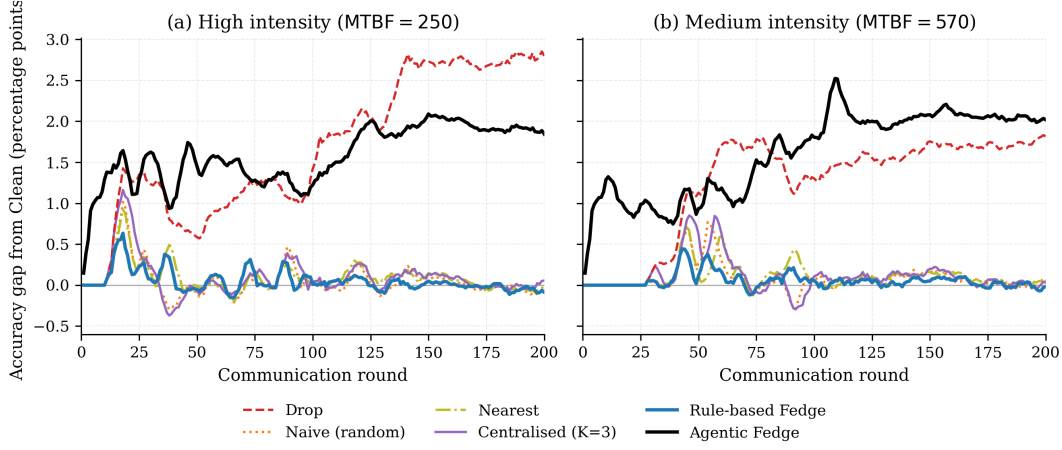


FIGURE 5.3: Per-round accuracy gap from the failure-free Clean upper bound for the six failover policies under the Scenario A configuration, at high (left, MTBF = 250) and medium (right, MTBF = 570) intensity. The y -axis reports $(\text{Clean} - \text{method}) \times 100$ in percentage points; the horizontal line at zero is the Clean reference. The Rule-based Fedge and the three recovery baselines hug the reference, Drop accumulates the largest persistent gap, and the Agentic Fedge tracks Drop more closely at high intensity, which is the global-accuracy signature of the stranded-clients result in Table 5.2.

ten agentic rounds carry an LLM-emitted, constraint-valid action with a logged rationale, and the remainder revert to a deterministic capacity-aware placement. System correctness is preserved by the validator across the full fallback distribution: across all Scenario A runs the four invariants of capacity, liveness, uniqueness, and self-preservation hold without violation. The fallback rate characterises the reliability of the LLM Coordinator under this model, not the correctness of the system.

5.4.3 Cross-Partition Failover

This subsection reports recovery from Fedge-node failover, the second failure class of Section 5.1.3, under the Scenario B configuration of Section 5.4.1. When a Fedge crashes, all of its edges are orphaned and must be hosted by surviving Fedges within the per-Fedge edge-capacity ceiling $\kappa_f^{\text{edge}} = 8$. Because no edge is reassigned by the Naive or Nearest client-level policies at the cluster scale, the comparison is reported against the four methods that act at the Fedge granularity: Drop, Centralised, the Rule-based Fedge, and the Agentic Fedge. Table 5.3 reports the operational metrics and Table 5.4 the agentic-specific diagnostics.

Decision latency and capacity binding. The Agentic Fedge attains $\Lambda = 0.00$ on every event by deliberating within the round of detection; the Rule-based Fedge attains 0.28 through precomputation, with the remaining events falling back to one-round reactive computation on instantaneous Fedge crashes; Centralised pays its three-round detection delay on every event. The capacity result at cluster scale exposes the complementary trade-offs of the two proposed designs. Centralised and the Agentic Fedge bind the per-Fedge edge ceiling exactly at $\kappa_f^{\text{edge}} = 8$ on every run. The Rule-based Fedge approaches the ceiling and briefly holds above it under cascade pressure (mean max load 8.8 ± 1.8 , with one run reaching 12). This is the inherent behaviour of distributed precomputation in which multiple Fedges commit

TABLE 5.3: Scenario B operational metrics (cluster cascade, $\kappa_f^{\text{edge}} = 8$, $T = 200$ rounds). Mean $\pm$ standard deviation over independent runs. Bold marks the best value per column among the capacity-respecting methods.

Method	$\Lambda \downarrow$ (rounds)	Migration (edges/event)	Max load $\downarrow$ (edges)	Real-time $\uparrow$ (frac.)	Stranded $\downarrow$ (edge-rds)	Audit
Drop	n/a	0	4.0 ± 0.0	n/a	2443 ± 356	no
Centralised	3.00 ± 0.00	5.12 ± 0.72	8.0 ± 0.0	0.00 ± 0.00	77 ± 11	no
Rule-based Fedge	0.28 ± 0.23	5.28 ± 0.91	8.8 ± 1.8	0.72 ± 0.23	290 ± 452	no
Agentic Fedge	0.00 ± 0.00	3.20 ± 0.98	8.0 ± 0.0	1.00 ± 0.00	545 ± 571	yes

Drop never migrates, so Λ and the real-time rate are undefined and the max load stays at the per-Fedge initial allocation. The Rule-based Fedge attains a low Λ through precomputation but exceeds the per-Fedge edge cap on a subset of runs (mean max load 8.8, with one run reaching 12); the constraint validator in the Agentic Fedge prevents this class of error by construction.

TABLE 5.4: Scenario B agentic-specific diagnostics (mean $\pm$ standard deviation over independent runs).

Diagnostic	Agentic Fedge
Tier-1 fast path (% rounds)	24.3 ± 9.4
Tier-2 deliberative (% rounds)	33.2 ± 12.1
Tier-1 fallback (% rounds)	42.5 ± 18.7
Total constraint violations	592 ± 342
Total LLM calls	398 ± 73
Tier-2 overhead p50 (ms)	$17,169 \pm 4,484$
Tier-2 overhead p95 (ms)	$21,241 \pm 3,349$
Final global accuracy	0.520 ± 0.019

their cached plans concurrently within the same round: each Fedge’s local topology view does not yet reflect the others’ commits, so the receiving Fedge can briefly hold above its nominal cap. The Rule-based design accepts this transient above-nominal load on the receiving Fedge as the cost of decisive cached-plan commit, which is the same property that delivers its zero-latency commit and its one-order-of-magnitude reduction in stranded edges below Drop. The Agentic Fedge takes the complementary trade-off: it re-plans at commit time on fresh telemetry through the LLM Coordinator and binds the ceiling strictly on every commit, at the cost of seconds of inference time per round. The two designs therefore make complementary trade-offs at the cluster scale rather than one dominating the other.

Stranded count and churn. Centralised has the lowest non-zero stranded count (77 ± 11 , the structural consequence of its fixed three-round detection window). The Rule-based Fedge sits at 290 ± 452 , with the high variance reflecting cascade-overlap rounds where its cached plan does not cover all simultaneous failures. The Agentic Fedge sits at 545 ± 571 , the cost of constraint-valid but coverage-incomplete LLM actions on cascade rounds, partly offset by the Tier-1 deterministic fallback. Drop accumulates two orders of magnitude more (2443 ± 356).

Tier-1 fallback rate and the operating range. The cluster-scale Tier-1 fallback rate is the central diagnostic of this scenario. Over all rounds it is $42.5\% \pm 18.7\%$; taken over only the escalated rounds (Tier-2 deliberative plus Tier-1 fallback, 75.7% of the episode) it is approximately 56%. Of the rounds in which the LLM Coordinator is asked to commit a cluster-level action, more than half hit the deterministic fallback; the validator catches the violations and the system stays correct, but the

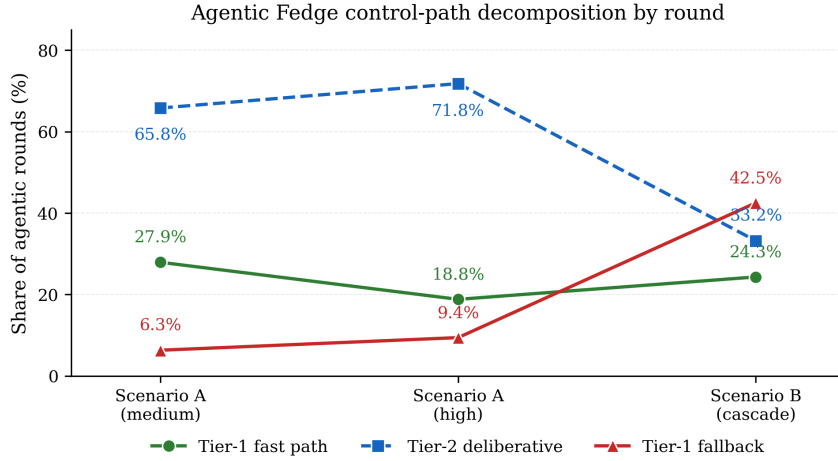


FIGURE 5.4: Round-resolution breakdown of the Agentic Fedge control path: the share of rounds resolved by the Tier-1 fast path, committed by Tier-2 deliberation, and committed by the Tier-1 deterministic fallback, for Scenario A (medium, high) and Scenario B (cascade). The fallback share rises from 6 to 9% at the edge scale to 42.5% at the cluster scale, which is the operating-range finding of Scenario B.

LLM’s contribution to those rounds is a single discarded attempt. This is materially higher than the 6 to 9% edge-level rate observed under Scenario A, for three structural reasons: cluster decisions involve an order of magnitude more entities, the cascade forces five Fedge failures into a narrow window so the prompt becomes more constrained as healthy capacity falls, and the larger cluster prompt concentrates JSON-schema validation failures on the more complex emission. The architectural claim that LLM unreliability is a parameter rather than a blocker holds empirically: capacity is never exceeded, no edge is placed on a failed Fedge, and no edge is double-placed. The operational claim that the agentic plane drives cluster-level decisions, however, holds only for the 33.2% of rounds where Tier-2 actually commits. Reported with both denominators, the Tier-1 fallback rate is the single number that defines the operating range of LLM-driven cross-partition orchestration on a 7B-class model: the cluster Coordinator at this model size is a stress test rather than a deployable component, while system correctness is preserved by the validator regardless.

5.4.4 Tier-3 Memory Ablation

This subsection isolates the contribution of the Tier-3 episodic memory store described in Section 5.3. The Agentic Fedge is rerun on the Scenario A high-intensity configuration with the memory tier disabled, leaving the Tier-1 triage and Tier-2 deliberative paths unchanged. With memory off, the Coordinator prompt no longer contains retrieved playbooks and the memory pre-check that forces escalation on previously-poor-outcome situations is inactive. All other components are held fixed (the failure schedule, the constraint validator V , the model and decoding configuration, and the prompt template). The two arms are run with an asymmetric allocation of ten memory-on runs against three memory-off runs, with the smaller memory-off arm justified by the agentic-specific effect sizes being large relative to the within-arm standard deviations. All thirteen runs complete the full 200-round training horizon. The memory-on arm in this ablation is a separate execution and is not the same set of

TABLE 5.5: Tier-3 memory ablation (Scenario A, high intensity, $T = 200$ rounds; ten memory-on runs, three memory-off runs). Mean $\pm$ standard deviation per arm; the rightmost column reports the per-metric difference $\Delta = (\text{off} - \text{on})$. Bold rows mark the metrics on which the two arms separate beyond noise.

Metric	Memory ON	Memory OFF	Δ (off-on)
<i>Tier mix (share of agentic rounds)</i>			
Tier-1 fast path (%)	15.2 $\pm$ 1.2	15.7 $\pm$ 0.3	+0.4
Tier-2 deliberative (%)	70.3 $\pm$ 10.9	77.0 $\pm$ 2.6	+6.7
Tier-1 fallback (%)	14.4 $\pm$ 11.0	7.3 $\pm$ 2.6	-7.1
<i>Validator interventions</i>			
Total constraint violations	242 $\pm$ 109	85 $\pm$ 23	-157
Total LLM calls	463 $\pm$ 34	390 $\pm$ 12	-73
<i>Training outcomes</i>			
Reroute events	53.2 $\pm$ 20.1	53.7 $\pm$ 17.2	+0.5
Total migrations	231.5 $\pm$ 50.6	230.7 $\pm$ 56.1	-0.8
Live edges at $T = 200$ (of 10)	5.0 $\pm$ 0.0	5.0 $\pm$ 0.0	$\pm$ 0.0
Final global accuracy	0.568 $\pm$ 0.009	0.569 $\pm$ 0.011	+0.001
Final personalised acc. (mean)	0.598 $\pm$ 0.016	0.607 $\pm$ 0.011	+0.008

runs as the Agentic Fedge row of Table 5.2; the two arms share their failure schedules with each other but not with the headline Scenario A results above.

The ablation is reported on three classes of metric, in the order Table 5.5 presents them: tier-mix metrics that diagnose where in the control path the agentic rounds resolve, validator-intervention metrics that count how often the LLM emits a constraint-violating action, and training-outcome metrics that test whether the choice of memory configuration leaves any visible signal on the trained model. Figure 5.5 pairs the tier mix with the per-run distribution of constraint violations.

Training outcomes are unchanged. The bottom block of Table 5.5 shows that the training-outcome metrics are statistically indistinguishable across the two arms. Final global accuracy differs by 0.001, per-edge personalised accuracy by 0.008 (well inside the standard deviation of either arm), the number of reroute events and total migrations match within noise, and the surviving edge count at $T = 200$ is exactly five in every run of both arms. The failure schedule, the recovery work, and the training outcome are not perturbed by the presence or absence of the memory tier.

LLM reliability is mildly degraded by memory. The agentic-specific metrics carry a different signal. Disabling the memory tier cuts the Tier-1 fallback rate roughly in half (from 14.4% to 7.3%) and the total number of constraint violations to about a third (from 242 to 85). The Coordinator with memory enabled emits more schema-violating actions and consequently consumes more LLM calls (463 vs 390) on corrective retries. The most plausible mechanism is a prompt-budget interaction at this model size: the retrieved playbooks lengthen the Coordinator prompt without adding signal that the 7B model can productively use, and push the structured-output emission closer to the schema-validation failure boundary. The wide variance on the memory-on arm (Tier-1 fallback $\sigma = 11.0$ against $\sigma = 2.6$ off; violations $\sigma = 109$ against $\sigma = 23$ off, both visible in Figure 5.5(b)) is consistent with this reading: some memory-on runs receive unlucky retrievals that compound through training, while the memory-off arm uses a uniform prompt skeleton on every call.

What the ablation is really testing. The result inverts the conventional expectation that an episodic-memory tier improves the agent’s operational quality, and the cleanest reading is architectural rather than memory-specific. The memory tier is the

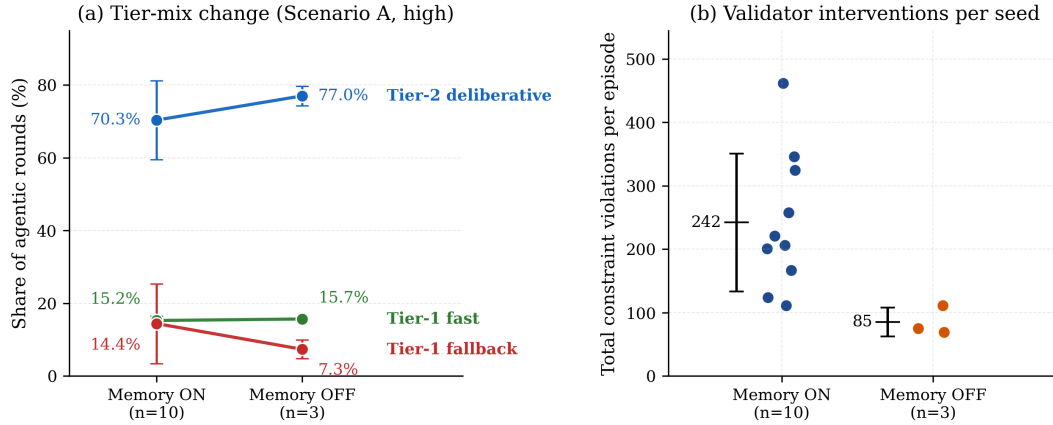


FIGURE 5.5: Memory ablation on Scenario A high intensity. (a) Tier mix as a share of all agentic rounds: disabling memory increases the share of rounds resolved by Tier-2 deliberation by 6.7 percentage points and halves the Tier-1 fallback share. (b) Per-run total constraint violations: memory-off runs cluster tightly around the arm mean (85 ± 23), whereas memory-on runs are higher in mean (242 ± 109) and span a much wider range. Black bars mark mean $\pm$ standard deviation.

mechanism that generates additional LLM unreliability in this ablation; the system as a whole remains correct because every committed action passes through the same constraint validator V of Section 5.1.4. Training outcomes are statistically indistinguishable across arms precisely because the Tier-1 fallback absorbs the LLM-side variance before it reaches the membership update. The ablation is therefore a stress test of the safety floor first and a test of memory second: the safety-floor invariants hold across a $3\times$ swing in LLM-emitted violations, and the training-relevant metrics remain unchanged within noise across the two arms. This is the cleanest single demonstration in the chapter that the architectural floor, comprising the validator and the Tier-1 fallback, is what carries system correctness, not any particular LLM-side component.

Two limitations should be noted about this result, both arising from the hardware available for this work. First, the two arms have unequal run counts (ten memory-on against three memory-off) and the experiment is run at only one failure intensity; a balanced allocation and a wider intensity sweep would reduce the variance of the agentic-specific metrics. Second, and more importantly, the deliberative path is limited to a 7B-class quantised model, because the L4 24 GB GPU available for this work cannot serve a larger model in real time at the round budget required. The most likely explanation for the inverted memory result is specific to models at this scale: the retrieved playbooks lengthen the Coordinator prompt, the prompt already carries a long structured payload, and the combination pushes the model closer to producing output that fails schema validation. Whether a larger model of ≥ 14 B parameters would show a positive contribution from the memory tier therefore cannot be answered with the present hardware. A repeat of the ablation at the ≥ 14 B scale, which requires A100-class compute beyond the L4 24 GB tier used here, is the principal follow-up. The architectural finding of this experiment is independent of either limitation: the safety-floor invariants hold despite the LLM-side variance that the memory tier introduces, and the training outcomes do not change with that variance.

5.5 Summary

HFL deployments on the edge-cloud continuum are exposed to continuous infrastructure pressure as they move from controlled experimental settings into production, and the orchestration layer that detects failures and reassigns workload across surviving components becomes central to system reliability. In this chapter, HFL orchestration was formulated as a distributed control problem with a unified failure model spanning within-partition edge failures and cross-partition Fedge-node failover, and a safety-floor architecture was proposed in which a constraint validator that gates every committed action and a deterministic Tier-1 fallback together preserve four invariants of capacity, liveness, uniqueness, and self-preservation. Two proactive orchestration designs were developed on this floor: a deterministic Rule-based Fedge that absorbs the reactive decision-latency cost through precomputation during the warning window, and an Agentic Fedge that places an LLM deliberative path behind the same triage and validator and logs a per-decision natural-language rationale for every committed action. Extensive experiments on CIFAR-10 across two failure scenarios and six reference orchestration policies showed that the Rule-based Fedge achieves zero stranding within a partition and constant-time commit, while the Agentic Fedge achieves zero-round decision latency and an auditable rationale channel at the cost of elevated stranded and churn figures under cascade pressure. A Tier-1 fallback rate of 6 to 9% at the edge scale of Scenario A and 42.5% at the cluster scale of Scenario B was reported as a new model-independent metric that isolates LLM-Coordinator reliability from system correctness, and a further ablation of the Tier-3 episodic memory store on the Scenario A high-intensity configuration found the training-relevant metrics statistically indistinguishable across the memory-on and memory-off arms, which is the strongest single demonstration that the architectural safety floor carries system correctness independent of any specific LLM-side component.

Chapter 6

Conclusion

6.1 Limitations and Future Work

6.1.1 Chapter 5: Agentic Orchestration

Two items bound the scope of the empirical evaluation reported above. Each is tied to a constraint on the experimental envelope rather than to the designs themselves.

The first is the model size of the deliberative path. The Tier-2 path is held at a 7B-class quantised model, because the L4 24 GB GPU available for this work cannot serve a larger one in real time at the round budget the experiments require. A repeat of the memory ablation at the ≥ 14 B scale is identified as the principal follow-up; the required A100-class compute is beyond the present hardware envelope, so it remains a future task. The architectural finding of the memory ablation is independent of model scale, and the memory tier is retained in the architecture as a designed capability whose empirical value at larger scale is therefore deferred.

The second is the coverage of the failure model. Failures in both scenarios are drawn from the per-node Poisson family fixed in Section 5.4.1. The motivating claim that an LLM-driven path can in principle reason about failure patterns outside a fixed rule table is therefore not directly tested in this chapter. An evaluation under out-of-distribution failure modes, in which the failure type is drawn from a held-out distribution at test time, is identified as the follow-up that would test this claim head-on.

Within these scopes, the evidence generated by the three experiments reported above is sufficient to establish the architectural safety-floor result of this chapter and to characterise the trade-off space between the two proposed designs.

Appendix A

Full Proof of the Fedge Convergence Theorem

This appendix gives the full derivation of Theorem 3.1 of Chapter 3. The structure of the appendix mirrors the four-step proof sketch of Section 3.4.4: a one-step descent inequality (Section A.1), a bound on local-update drift (Section A.2), the variance bound on the membership-weighted aggregate (Section A.3), and the telescoping argument (Section A.4). The notation of Chapter 3 is used throughout; in particular, w_k^t denotes the cloud-tier model for cluster k at round t , $\Pi_{e,k}^t$ the soft membership of edge e in cluster k , $\mathcal{S}_e^t$ the participating subset at edge e , $n_e^t = |\mathcal{S}_e^t|$, $C = |\mathcal{C}_e|$ (taken uniform), $\rho_e^t = n_e^t/C$, and $\bar{\rho} = \frac{1}{M} \sum_{e=1}^M \rho_e^t$.

The proof is conducted on the per-cluster objective

$$F_k(w) = \frac{\sum_{e=1}^M \Pi_{e,k}^* |\mathcal{C}_e| F_e(w)}{\sum_{e=1}^M \Pi_{e,k}^* |\mathcal{C}_e|}, \quad (\text{A.1})$$

where Π^* is the stabilised soft membership matrix after round T_0 (Assumption A5). The proof bounds $\mathbb{E} \|\nabla F_k(w_k^t)\|^2$ averaged over the post-warm-up rounds $t = T_0, \dots, T-1$ and over clusters $k = 1, \dots, K^*$.

A.1 One-Step Descent Inequality

Lemma A.1 (Descent inequality). *Under Assumption A1, the iterates of the Fedge algorithm satisfy*

$$\mathbb{E}[F_k(w_k^{t+1})] \leq F_k(w_k^t) - \eta \langle \nabla F_k(w_k^t), \mathbb{E}[\tilde{g}_k^t] \rangle + \frac{L\eta^2}{2} \mathbb{E} \|\tilde{g}_k^t\|^2, \quad (\text{A.2})$$

where $\tilde{g}_k^t$ is the aggregated update applied to cluster k at round t and η is the local step size.

Proof. The update rule of cluster k from Equation (3.12) of Chapter 3 can be written as $w_k^{t+1} = w_k^t - \eta \tilde{g}_k^t$, with $\tilde{g}_k^t$ the membership-weighted aggregated update. Applying Assumption A1 yields

$$F_k(w_k^{t+1}) \leq F_k(w_k^t) + \langle \nabla F_k(w_k^t), w_k^{t+1} - w_k^t \rangle + \frac{L}{2} \|w_k^{t+1} - w_k^t\|^2. \quad (\text{A.3})$$

Substituting the update rule and taking expectations over the stochastic gradient noise and the participation sampling gives Equation (A.2). $\square$

A.2 Bound on Local-Update Drift

The local SGD iterates at client $i \in \mathcal{S}_e^t$ drift from the blended model $\tilde{w}_e^t$ over the τ local steps. The drift bound below extends the SCAFFOLD-style analysis of [22] to the hierarchical setting with soft membership.

Lemma A.2 (Local-update drift). *Under Assumptions A1, A2, and A3, the local iterates of client $i \in \mathcal{S}_e^t$ at local step $s \in \{1, \dots, \tau\}$ satisfy*

$$\mathbb{E} \|w_{i,s}^t - \tilde{w}_e^t\|^2 \leq 2\eta^2 \tau s (\sigma^2 + 2G_{\text{intra}}^2), \quad s = 1, \dots, \tau, \quad (\text{A.4})$$

provided $\eta L \tau \leq 1/2$.

Proof. The local update at step s is $w_{i,s+1}^t = w_{i,s}^t - \eta \nabla F_i(w_{i,s}^t; \xi_{i,s})$ with $w_{i,0}^t = \tilde{w}_e^t$. Expanding the squared distance,

$$\mathbb{E} \|w_{i,s+1}^t - \tilde{w}_e^t\|^2 = \mathbb{E} \|w_{i,s}^t - \tilde{w}_e^t - \eta \nabla F_i(w_{i,s}^t; \xi_{i,s})\|^2. \quad (\text{A.5})$$

Decomposing the inner term using $\nabla F_i(w_{i,s}^t; \xi_{i,s}) = \nabla F_i(\tilde{w}_e^t) + (\nabla F_i(w_{i,s}^t) - \nabla F_i(\tilde{w}_e^t)) + (\nabla F_i(w_{i,s}^t; \xi_{i,s}) - \nabla F_i(w_{i,s}^t))$ and applying Assumption A2 to the stochastic term, Assumption A1 to the deterministic difference, and Assumption A3 to bound $\|\nabla F_i(\tilde{w}_e^t) - \nabla F_e(\tilde{w}_e^t)\|^2 \leq G_{\text{intra}}^2$, the recursion

$$\mathbb{E} \|w_{i,s+1}^t - \tilde{w}_e^t\|^2 \leq (1 + \eta L) \mathbb{E} \|w_{i,s}^t - \tilde{w}_e^t\|^2 + 2\eta^2 (\sigma^2 + 2G_{\text{intra}}^2), \quad (\text{A.6})$$

is obtained. Iterating from $s = 0$ and using $(1 + \eta L)^\tau \leq 2$ when $\eta L \tau \leq 1/2$ yields Equation (A.4). $\square$

A.3 Variance of the Membership-Weighted Aggregate

This section establishes the variance bound that constitutes the technical novelty of the proof. The aggregated update for cluster k at round t is

$$\tilde{g}_k^t = \frac{\sum_{e=1}^M \Pi_{e,k}^* n_e^t \Delta w_e^t}{\sum_{e=1}^M \Pi_{e,k}^* n_e^t}, \quad (\text{A.7})$$

where Δw_e^t is the edge-tier update produced by edge e in round t . The denominator of Equation (A.7) is denoted by $S_k^t = \sum_{e=1}^M \Pi_{e,k}^* n_e^t$, the effective participating sample size in cluster k .

Lemma A.3 (Variance of the membership-weighted aggregate). *Under Assumptions A1–A4, the variance of $\tilde{g}_k^t$ around the cluster gradient $\nabla F_k(w_k^t)$ satisfies*

$$\mathbb{E} \|\tilde{g}_k^t - \nabla F_k(w_k^t)\|^2 \leq \frac{\sigma^2 + 4G_{\text{intra}}^2}{S_k^t} + \frac{\gamma G_{\text{inter}}^2}{K^*} + \mathcal{O}(\eta^2 L^2 \tau^2 (\sigma^2 + G_{\text{intra}}^2)). \quad (\text{A.8})$$

Proof. The variance is decomposed via three triangle-inequality steps. Define $\tilde{g}_e^t = \nabla F_e(\tilde{w}_e^t)$ as the edge-tier gradient at the blended model and $\bar{g}_k^t = \frac{1}{S_k^t} \sum_{e=1}^M \Pi_{e,k}^* n_e^t \tilde{g}_e^t$ as its membership-weighted average. Then

$$\|\tilde{g}_k^t - \nabla F_k(w_k^t)\|^2 \leq 3\|\tilde{g}_k^t - \bar{g}_k^t\|^2 + 3\|\bar{g}_k^t - \nabla F_k(w_k^t)\|^2 + 3\|\nabla F_k(\tilde{w}_k^t) - \nabla F_k(w_k^t)\|^2, \quad (\text{A.9})$$

where $\tilde{w}_k^t = \frac{1}{S_k^t} \sum_{e=1}^M \Pi_{e,k}^* n_e^t \tilde{w}_e^t$ is the membership-weighted blend over cluster k . The three terms are bounded in turn.

Stochastic and intra-edge term. The first term $\|\tilde{g}_k^t - \bar{g}_k^t\|^2$ is the sample-average noise of the edge-aggregated updates around their expectations. The edge-tier update Δw_e^t is itself an average over n_e^t participating clients' SGD trajectories, so its variance around $\bar{g}_e^t$ is at most $(\sigma^2 + 2G_{\text{intra}}^2)/n_e^t$ by Assumptions A2 and A3, plus the local-drift contribution of order $\eta^2 L^2 \tau^2 (\sigma^2 + G_{\text{intra}}^2)$ from Lemma A.2. Aggregating across edges with weights $\Pi_{e,k}^* n_e^t / S_k^t$ and applying the standard variance-of-weighted-mean identity yields

$$\mathbb{E} \|\tilde{g}_k^t - \bar{g}_k^t\|^2 \leq \frac{\sigma^2 + 2G_{\text{intra}}^2}{S_k^t} + \mathcal{O}(\eta^2 L^2 \tau^2 (\sigma^2 + G_{\text{intra}}^2)). \quad (\text{A.10})$$

Inter-cluster residual. The second term $\|\tilde{g}_k^t - \nabla F_k(\tilde{w}_k^t)\|^2$ is the gap between the membership-weighted average of edge gradients and the cluster gradient. By the definition of F_k in Equation (A.1) and the definition of ∇F_e as the unweighted average of ∇F_i over clients of edge e , this gap is bounded by the gradient dissimilarity between edges sharing cluster k :

$$\mathbb{E} \|\tilde{g}_k^t - \nabla F_k(\tilde{w}_k^t)\|^2 \leq \frac{1}{S_k^t} \sum_{e=1}^M \Pi_{e,k}^* n_e^t \|\bar{g}_e^t - \nabla F_k(\tilde{w}_k^t)\|^2 \leq \zeta_k^2, \quad (\text{A.11})$$

where the first inequality is Jensen's inequality applied to the convex map $\|\cdot\|^2$ on the membership-weighted simplex, and the second inequality is the definition of ζ_k^2 from Assumption A4. Averaging over k and applying Assumption A4,

$$\frac{1}{K^*} \sum_{k=1}^{K^*} \mathbb{E} \|\tilde{g}_k^t - \nabla F_k(\tilde{w}_k^t)\|^2 \leq \gamma G_{\text{inter}}^2. \quad (\text{A.12})$$

The factor of $1/K^*$ appears once Equation (A.12) is substituted back into the per-cluster bound through the averaging in Equation (3.28).

Smoothness term. The third term $\|\nabla F_k(\tilde{w}_k^t) - \nabla F_k(w_k^t)\|^2$ is bounded using Assumption A1 and the drift bound of Lemma A.2:

$$\mathbb{E} \|\nabla F_k(\tilde{w}_k^t) - \nabla F_k(w_k^t)\|^2 \leq L^2 \mathbb{E} \|\tilde{w}_k^t - w_k^t\|^2 \leq \mathcal{O}(\eta^2 L^2 \tau^2 (\sigma^2 + G_{\text{intra}}^2)). \quad (\text{A.13})$$

Combining Equations (A.10), (A.12), and (A.13), and using $S_k^t \geq \bar{\rho}C$ (which holds for any cluster with positive membership weight), yields Equation (A.8). $\square$

A.4 Telescoping over Post-Warm-Up Rounds

The proof is completed by telescoping the descent inequality of Lemma A.1 from round T_0 to round $T - 1$ and substituting the variance bound of Lemma A.3.

Proof of Theorem 3.1. By Lemma A.1, the descent inequality for cluster k at round $t \geq T_0$ reads

$$\mathbb{E}[F_k(w_k^{t+1})] \leq F_k(w_k^t) - \eta \langle \nabla F_k(w_k^t), \mathbb{E}[\tilde{g}_k^t] \rangle + \frac{L\eta^2}{2} \mathbb{E} \|\tilde{g}_k^t\|^2. \quad (\text{A.14})$$

Using the identity $\|\tilde{g}_k^t\|^2 = \|\tilde{g}_k^t - \nabla F_k(w_k^t)\|^2 + \|\nabla F_k(w_k^t)\|^2 + 2\langle \tilde{g}_k^t - \nabla F_k(w_k^t), \nabla F_k(w_k^t) \rangle$ and Young's inequality on the inner-product term,

$$\mathbb{E}[F_k(w_k^{t+1})] \leq F_k(w_k^t) - \frac{\eta}{2} \|\nabla F_k(w_k^t)\|^2 + \frac{\eta}{2} \mathbb{E}\|\tilde{g}_k^t - \nabla F_k(w_k^t)\|^2 + \frac{L\eta^2}{2} \mathbb{E}\|\tilde{g}_k^t\|^2. \quad (\text{A.15})$$

Rearranging and summing over $t = T_0, \dots, T-1$ yields

$$\frac{\eta}{2} \sum_{t=T_0}^{T-1} \mathbb{E}\|\nabla F_k(w_k^t)\|^2 \leq F_k(w_k^{T_0}) - F_k^* + \frac{\eta}{2} \sum_{t=T_0}^{T-1} \mathbb{E}\|\tilde{g}_k^t - \nabla F_k(w_k^t)\|^2 + \frac{L\eta^2}{2} \sum_{t=T_0}^{T-1} \mathbb{E}\|\tilde{g}_k^t\|^2, \quad (\text{A.16})$$

where $F_k^* = \inf_w F_k(w)$. Substituting the variance bound from Lemma A.3 and bounding $\|\tilde{g}_k^t\|^2 \leq 2\|\tilde{g}_k^t - \nabla F_k(w_k^t)\|^2 + 2\|\nabla F_k(w_k^t)\|^2$, then averaging over clusters $k = 1, \dots, K^*$, gives

$$\begin{aligned} \frac{1}{T-T_0} \sum_{t=T_0}^{T-1} \frac{1}{K^*} \sum_{k=1}^{K^*} \mathbb{E}\|\nabla F_k(w_k^t)\|^2 &\leq \frac{2\Delta_0}{\eta(T-T_0)} + (1+2L\eta) \frac{\sigma^2 + 4G_{\text{intra}}^2}{\bar{\rho}C} \\ &\quad + (1+2L\eta) \frac{\gamma G_{\text{inter}}^2}{K^*} + \mathcal{O}(\eta^2 L^2 \tau^2 (\sigma^2 + G_{\text{intra}}^2)), \end{aligned} \quad (\text{A.17})$$

where $\Delta_0 = F(w^{T_0}) - F^*$.

Choosing the step size $\eta = \min(1/(2L\tau), 1/(L\sqrt{(T-T_0)\bar{\rho}C}))$ balances the dominant terms. Substituting this step size and absorbing constants gives

$$\frac{1}{T-T_0} \sum_{t=T_0}^{T-1} \frac{1}{K^*} \sum_{k=1}^{K^*} \mathbb{E}\|\nabla F_k(w_k^t)\|^2 \leq \mathcal{O}\left(\frac{L\Delta_0 + \sigma^2}{\sqrt{(T-T_0)\bar{\rho}C}}\right) + \mathcal{O}\left(\frac{\gamma G_{\text{inter}}^2}{K^*}\right) + \mathcal{O}\left(\frac{G_{\text{intra}}^2}{\bar{\rho}C}\right), \quad (\text{A.18})$$

which is the bound of Theorem 3.1. $\square$

A.5 Notes on the Constants

The constants hidden in the $\mathcal{O}$ notation of Equation (A.18) depend only on the universal constants of the analysis (the constant L of Assumption A1, numerical factors arising from Young's and Jensen's inequalities, and the step-size constant) and are independent of the data partition, the number of edges M , the number of clusters K^* , the participation rate ρ_e^t , and the local step count τ . The bound therefore scales correctly with the system parameters and supports the corollaries of Section 3.4.5.

The optimal cluster count K_{opt}^* of Corollary 3.3 is obtained by differentiating the right-hand side of Equation (A.18) with respect to K^* and setting the derivative to zero. The two terms that depend on K^* are term (i), through the implicit cost of small per-cluster sample size when K^* is large, and term (ii), which decreases as $1/K^*$. Equating the marginal rates and solving for K^* yields Equation (3.35).

Bibliography

- [1] Mahadev Satyanarayanan. “The Emergence of Edge Computing”. In: *Computer* 50.1 (2017), pp. 30–39. DOI: [10.1109/MC.2017.9](https://doi.org/10.1109/MC.2017.9).
- [2] Weisong Shi et al. “Edge Computing: Vision and Challenges”. In: *IEEE Internet of Things Journal* 3.5 (2016), pp. 637–646. DOI: [10.1109/JIOT.2016.2579198](https://doi.org/10.1109/JIOT.2016.2579198).
- [3] Shuiguang Deng et al. “Edge Intelligence: The Confluence of Edge Computing and Artificial Intelligence”. In: *IEEE Internet of Things Journal* 7.8 (2020), pp. 7457–7469. DOI: [10.1109/JIOT.2020.2984887](https://doi.org/10.1109/JIOT.2020.2984887).
- [4] Panagiotis Gkonis et al. “A Survey on IoT–Edge–Cloud Continuum Systems: Status, Challenges, Use Cases, and Open Issues”. In: *Future Internet* 15.12 (2023), p. 383. DOI: [10.3390/fi15120383](https://doi.org/10.3390/fi15120383).
- [5] Daniel Mota-Cruz et al. “Navigating the Edge-Cloud Continuum: A State-of-Practice Survey”. In: *arXiv preprint arXiv:2506.02003* (2025). arXiv: [2506.02003](https://arxiv.org/abs/2506.02003).
- [6] Amine Abouaoumar et al. “Resource Provisioning in Edge Computing for Latency-Sensitive Applications”. In: *IEEE Internet of Things Journal* 8.14 (2021), pp. 11088–11099. DOI: [10.1109/JIOT.2021.3052082](https://doi.org/10.1109/JIOT.2021.3052082).
- [7] Flavio Bonomi et al. “Fog Computing and Its Role in the Internet of Things”. In: *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*. Helsinki, Finland: ACM, 2012, pp. 13–16. DOI: [10.1145/2342509.2342513](https://doi.org/10.1145/2342509.2342513).
- [8] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data, and Repealing Directive 95/46/EC (General Data Protection Regulation)*. Official Journal of the European Union, L 119, pp. 1–88. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [9] Peter Kairouz et al. “Advances and Open Problems in Federated Learning”. In: *Foundations and Trends in Machine Learning* 14.1–2 (2021), pp. 1–210. DOI: [10.1561/22000000083](https://doi.org/10.1561/22000000083).
- [10] H. Brendan McMahan et al. “Communication-Efficient Learning of Deep Networks from Decentralized Data”. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 54. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1273–1282.
- [11] Enrique Tomás Martínez Beltrán et al. “Decentralized Federated Learning: Fundamentals, State of the Art, Frameworks, Trends, and Challenges”. In: *IEEE Communications Surveys & Tutorials* 25.4 (2023), pp. 2983–3013. DOI: [10.1109/COMST.2023.3315746](https://doi.org/10.1109/COMST.2023.3315746).
- [12] Ligeng Zhu, Zhijian Liu, and Song Han. “Deep Leakage from Gradients”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019.

- [13] Lumin Liu et al. "Client-Edge-Cloud Hierarchical Federated Learning". In: *ICC 2020 – 2020 IEEE International Conference on Communications*. Dublin, Ireland: IEEE, 2020, pp. 1–6. DOI: [10.1109/ICC40277.2020.9148862](https://doi.org/10.1109/ICC40277.2020.9148862).
- [14] Qiong Wu et al. "HiFlash: Communication-Efficient Hierarchical Federated Learning with Adaptive Staleness Control and Heterogeneity-Aware Client-Edge Association". In: *IEEE Transactions on Parallel and Distributed Systems* 34.5 (2023), pp. 1560–1579. DOI: [10.1109/TPDS.2023.3257541](https://doi.org/10.1109/TPDS.2023.3257541).
- [15] Chunjiong Gu and Baochun Li. "Hierarchical Federated Learning Architectures for the Metaverse". In: *ZTE Communications* 22.2 (2024), pp. 39–48. DOI: [10.12142/ZTECOM.202402006](https://doi.org/10.12142/ZTECOM.202402006).
- [16] Christopher Briggs, Zhong Fan, and Peter Andras. "Federated Learning with Hierarchical Clustering of Local Updates to Improve Training on Non-IID Data". In: *2020 International Joint Conference on Neural Networks (IJCNN)*. Glasgow, United Kingdom: IEEE, 2020, pp. 1–9. DOI: [10.1109/IJCNN48605.2020.9207469](https://doi.org/10.1109/IJCNN48605.2020.9207469).
- [17] Yue Zhao, Meng Li, Liangzhen Lai, et al. "Federated learning with non-IID data". In: *arXiv preprint arXiv:1806.00582* (2018).
- [18] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. "Measuring the effects of non-identical data distribution for federated visual classification". In: *NeurIPS Workshop on Federated Learning*. 2019.
- [19] Chuan Chen et al. "Advances in Robust Federated Learning: Heterogeneity Considerations". In: *arXiv preprint arXiv:2405.09839* (2024).
- [20] Md Palash Uddin et al. "A Systematic Literature Review of Robust Federated Learning: Issues, Solutions, and Future Research Directions". In: *ACM Computing Surveys* 57.10 (2025), pp. 1–38. DOI: [10.1145/3727643](https://doi.org/10.1145/3727643).
- [21] Tian Li et al. "Federated optimization in heterogeneous networks". In: *Proceedings of Machine learning and systems* 2 (2020), pp. 429–450.
- [22] Sai Praneeth Karimireddy et al. "Scaffold: Stochastic controlled averaging for federated learning". In: *International conference on machine learning*. PMLR. 2020, pp. 5132–5143.
- [23] Jianyu Wang et al. "Tackling the objective inconsistency problem in heterogeneous federated optimization". In: *Advances in neural information processing systems* 33 (2020), pp. 7611–7623.
- [24] Durmus Alp Emre Acar et al. "Federated learning based on dynamic regularization". In: *arXiv preprint arXiv:2111.04263* (2021).
- [25] Sashank Reddi et al. "Adaptive federated optimization". In: *arXiv preprint arXiv:2003.00295* (2020).
- [26] Canh T Dinh, Nguyen Tran, and Josh Nguyen. "Personalized federated learning with Moreau envelopes". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 21394–21405.
- [27] Alireza Fallah, Aryan Mokhtari, and Asuman Ozdaglar. "Personalized federated learning: A meta-learning approach". In: *arXiv preprint arXiv:2002.07948* (2020).
- [28] Virginia Smith et al. "Federated multi-task learning". In: *Advances in neural information processing systems* 30 (2017).

- [29] Krishna Pillutla et al. "Federated learning with partial model personalization". In: *Proc. Int. Conf. Mach. Learn. (ICML)*. 2022, pp. 17716–17758.
- [30] Xiaoxiao Li et al. "Fedbn: Federated learning on non-iid features via local batch normalization". In: *arXiv preprint arXiv:2102.07623* (2021).
- [31] Felix Sattler, Klaus-Robert Müller, and Wojciech Samek. "Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints". In: *IEEE transactions on neural networks and learning systems* 32.8 (2020), pp. 3710–3722.
- [32] Avishek Ghosh et al. "An efficient framework for clustered federated learning". In: *IEEE Transactions on Information Theory* 68.12 (2022), pp. 8076–8091.
- [33] Yichen Ruan and Carlee Joe-Wong. "FedSoft: Soft Clustered Federated Learning with Proximal Local Updating". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 36. 2022, pp. 8124–8131.
- [34] Wenzhi Fang et al. "Hierarchical federated learning with multi-timescale gradient correction". In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 78863–78904.
- [35] Chunmei Ma et al. "Personalized client-edge-cloud hierarchical federated learning in mobile edge computing". In: *Journal of Cloud Computing* 13.1 (2024), p. 161.
- [36] Noah Thomas Ottou Omgba et al. "AdaptiveHierFL: Cloud-Native Dynamic Multi-Tier Federated Learning with Adaptive Knowledge Transfer". In: *Brice Christian and Befoum, Stephane Richard, AdaptiveHierFL: Cloud-Native Dynamic Multi-Tier Federated Learning with Adaptive Knowledge Transfer (December 12, 2024)* (2024).
- [37] Seunghyun Lee, Shuaijun Chen, et al. "Personalizing Federated Learning for Hierarchical Edge Networks with Non-IID Data". In: *arXiv preprint arXiv:2504.08872* (2025).
- [38] Chen Qiu et al. "Hierarchical Aggregation for Federated Learning in Heterogeneous IoT Scenarios: Enhancing Privacy and Communication Efficiency." In: *Future Internet* 17.1 (2025).
- [39] Lei Fu et al. "Client selection in federated learning: Principles, challenges, and opportunities". In: *IEEE Internet of Things Journal* (2023).
- [40] Samara Mayhoub and Tareq M. Shami. "A review of client selection methods in federated learning". In: *Archives of Computational Methods in Engineering* 31.2 (2024), pp. 1129–1152.
- [41] J. Kim, H. Park, and M. Choi. "Federated Client Selection for Efficient Distributed Learning". In: *Proceedings of the IEEE International Conference on Distributed Computing (ICDC)*. 2023.
- [42] Zhe Qu et al. "Context-aware online client selection for hierarchical federated learning". In: *IEEE Transactions on Parallel and Distributed Systems* 33.12 (2022), pp. 4353–4367.
- [43] Ivan Čilić et al. "Reactive Orchestration for Hierarchical Federated Learning Under a Communication Cost Budget". In: *arXiv preprint arXiv:2412.03385* (2024). arXiv: 2412.03385.
- [44] Anna Lackinger et al. "Inference Load-Aware Orchestration for Hierarchical Federated Learning". In: *arXiv preprint arXiv:2407.16836* (2024). arXiv: 2407.16836.

- [45] Satwat Bashir et al. "Architectural Blueprint for Heterogeneity-Resilient Federated Learning". In: *IET 6G and Future Networks Conference (IET 6G 2024)*. London, UK, 2024, pp. 62–68. DOI: [10.1049/icp.2024.2236](https://doi.org/10.1049/icp.2024.2236).
- [46] Xiaohan Jiang and Hongbin Zhu. "On the Convergence of Hierarchical Federated Learning with Partial Worker Participation". In: *Proceedings of Machine Learning Research* 244 (2024). AISTATS 2024. URL: <https://proceedings.mlr.press/v244/jiang24a.html>.
- [47] Michael Crawshaw and Mingrui Liu. "Federated Learning under Periodic Client Participation and Heterogeneous Data: A New Communication-Efficient Algorithm and Analysis". In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2410.23131. 2024.
- [48] Paul Mangold et al. "SCAFFOLD with Stochastic Gradients: New Analysis with Linear Speed-Up". In: *arXiv preprint arXiv:2503.07594* (2025).
- [49] Lihua Song et al. "CHPFL: Clustered Adaptive Hierarchical Federated Learning for Edge-Level Personalization". In: *Internet of Things and Cyber-Physical Systems* (2025). DOI: [10.1016/j.iotcps.2025.04.002](https://doi.org/10.1016/j.iotcps.2025.04.002).
- [50] Sabtain Ahmad et al. "Clustered Federated Learning with Hierarchical Knowledge Distillation". In: *arXiv preprint arXiv:2512.10443* (2025).
- [51] Chaoqun You et al. "Hierarchical Personalized Federated Learning Over Massive Mobile Edge Computing Networks". In: *IEEE Transactions on Wireless Communications* (2023). arXiv:2303.10580; DOI: [10.1109/TWC.2023.3260141](https://doi.org/10.1109/TWC.2023.3260141).
- [52] Zhengjie Yang et al. "Hierarchical Federated Learning with Momentum Acceleration in Multi-Tier Networks". In: *IEEE Transactions on Parallel and Distributed Systems* 34.10 (2023), pp. 2629–2641.
- [53] Thomas S. Ferguson. "A Bayesian Analysis of Some Nonparametric Problems". In: *The Annals of Statistics* 1.2 (1973), pp. 209–230.
- [54] Mariona Jaramillo-Civill, Peng Wu, and Pau Closas. "DPMM-CFL: Clustered Federated Learning via Dirichlet Process Mixture Model Nonparametric Clustering". In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Accepted; arXiv:2510.07132. 2026.
- [55] Peng Wu, Tales Imbiriba, and Pau Closas. "A Bayesian Framework for Clustered Federated Learning". In: *arXiv preprint arXiv:2410.15473* (2024). Also appears in *IEEE Transactions*; full author affiliation: Northeastern University and UMass Boston.
- [56] Fu Peng and Ming Tang. "FedDAA: Dynamic Client Clustering for Concept Drift Adaptation in Federated Learning". In: *arXiv preprint arXiv:2506.21054* (2025).
- [57] Sonia Jain and Radford M. Neal. "A Split-Merge Markov Chain Monte Carlo Procedure for the Dirichlet Process Mixture Model". In: *Journal of Computational and Graphical Statistics* 13.1 (2004), pp. 158–182.
- [58] Brendan Burns et al. "Borg, Omega, and Kubernetes". In: *Communications of the ACM* 59.5 (2016), pp. 50–57. DOI: [10.1145/2890784](https://doi.org/10.1145/2890784).
- [59] Tushar Deepak Chandra and Sam Toueg. "Unreliable Failure Detectors for Reliable Distributed Systems". In: *Journal of the ACM* 43.2 (1996), pp. 225–267. DOI: [10.1145/226643.226647](https://doi.org/10.1145/226643.226647).

- [60] Naohiro Hayashibara et al. "The ϕ Accrual Failure Detector". In: *Proc. 23rd IEEE Int. Symp. Reliable Distributed Systems (SRDS)*. IEEE Computer Society, 2004, pp. 66–78. DOI: [10.1109/RELDIS.2004.1353004](https://doi.org/10.1109/RELDIS.2004.1353004).
- [61] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *Proc. Int. Conf. Learning Representations (ICLR)*. 2023.
- [62] Qingyun Wu et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation". In: *arXiv preprint arXiv:2308.08155* (2023).
- [63] Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.
- [64] Brandon T. Willard and Rémi Louf. "Efficient Guided Generation for Large Language Models". In: *arXiv preprint arXiv:2307.09702* (2023).
- [65] Pydantic Team. *Pydantic: Data Validation Using Python Type Hints*. <https://docs.pydantic.dev/>. Version 2.x. 2024.
- [66] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd ed. John Wiley & Sons, 2009.
- [67] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks". In: *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*. 2019, pp. 3982–3992. DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410).
- [68] An Yang et al. "Qwen2.5 Technical Report". In: *arXiv preprint arXiv:2412.15115* (2024).
- [69] Ji Lin et al. "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration". In: *Proc. Mach. Learn. Syst. (MLSys)*. 2024.
- [70] Woosuk Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*. 2023, pp. 611–626. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).